

第 10 章 数学概念与方法

学习目标

- ☑ 熟练掌握扩展欧几里德算法和它的时间复杂度
- ☑ 熟练掌握用筛法构造素数表，了解素数定理
- ☑ 学会求二元线性不定方程的整数解
- ☑ 熟练掌握模运算规则、快速幂取模算法和模线性方程的解法
- ☑ 熟悉杨辉三角、二项式定理和组合数的基本性质
- ☑ 学会推导约数个数公式和欧拉函数公式
- ☑ 熟练掌握可重集全排列的编码和解码算法
- ☑ 理解样本空间、事件和概率，学会用组合计数的方法计算离散概率
- ☑ 理解条件概率的概念和计算方法
- ☑ 理解连续概率和数学期望的概念和计算方法
- ☑ 熟悉常见计数序列，如 Fibonacci 数列、Catalan 数列等
- ☑ 熟悉建立递推关系的基本方法、常见错误和实现技巧

没有数学就没有算法；没有好的数学基础，也很难在算法上有所成就。本章介绍算法竞赛中涉及的常见数学概念和方法，包括数论、排列组合、递推关系和离散概率等。

10.1 数论初步

数论被“数学王子”高斯誉为整个数学王国的皇后。在算法竞赛中，数论常常以各种面貌出现，但万变不离其宗，大部分数论题目并不涉及多少特殊的知识，但对数学思维和能力要求较高。本节介绍几个最为常用的算法，并通过例题展示一些常用的思维方式。

10.1.1 欧几里德算法和唯一分解定理

除法表达式。给出一个这样的除法表达式： $X_1 / X_2 / X_3 / \cdots / X_k$ ，其中 X_i 是正整数。除法表达式应当按照从左到右的顺序求和，例如，表达式 $1/2/1/2$ 的值为 $1/4$ 。但可以在表达式中嵌入括号以改变计算顺序，例如，表达式 $(1/2)/(1/2)$ 的值为 1 。

输入 $X_1, X_2, \dots, X_k$ ，判断是否可以通过添加括号，使表达式的值为整数。 $K \leq 10000$ ， $X_i \leq 10^9$ 。

【分析】

表达式的值一定可以写成 A/B 的形式： A 是其中一些 X_i 的乘积，而 B 是其他数的乘积。

不难发现, X_2 必须放在分母位置, 那其他数呢?

幸运的是, 其他数都可以在分子位置:

$$E = X_1 / (X_2 / X_3 / \cdots X_k) = \frac{X_1 X_3 X_4 \cdots X_k}{X_2}$$

接下来的问题就变成了: 判断 E 是否为整数。

第1种方法是利用前面介绍的高精度运算: k 次乘法加一次除法。显然, 这个方法是正确的, 但却比较麻烦。

第2种方法是利用唯一分解定理, 把 X_2 写成若干素数相乘的形式:

$$X_2 = p_1^{a_1} p_2^{a_2} p_3^{a_3} \cdots$$

然后依次判断每个 $p_i^{a_i}$ 是否是 $X_1 X_3 X_4 \cdots X_k$ 的约数。这次不用高精度乘法了, 只需把所有 X_i 中 p_i 的指数加起来。如果结果比 a_i 小, 说明还会有 p_i 约不掉, 因此 E 不是整数。这种方法在第5章中已经用过, 这里不再赘述。

第3种方法是直接约分: 每次约掉 X_i 和 X_2 的最大公约数 $\gcd(X_i, X_2)$, 则当且仅当约分结束后 $X_2=1$ 时 E 为整数, 程序如下:

```
int judge(int* X) {
    X[2] /= gcd(X[2], X[1]);
    for(int i = 3; i <= k; i++) X[2] /= gcd(X[i], X[2]);
    return X[2] == 1;
}
```

整个算法的时间效率取决于这里的 $\gcd$ 算法。尽管依次试除也能得到正确的结果, 但还有一个简单、高效, 而且相当优美的算法——辗转相除法。它也许是最广为人知的数论算法。

辗转相除法的关键在于如下恒等式: $\gcd(a, b) = \gcd(b, a \bmod b)$ 。它和边界条件 $\gcd(a, 0)=a$ 一起构成了下面的程序:

```
int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}
```

这个算法称为欧几里德算法 (Euclid algorithm)。既然是递归, 那么免不了问一句: 会栈溢出吗? 答案是不会。可以证明, $\gcd$ 函数的递归层数不超过 $4.785 \lg N + 1.6723$, 其中 $N = \max\{a, b\}$ 。值得一提的是, 让 $\gcd$ 递归层数最多的是 $\gcd(F_n, F_{n-1})$, 其中 F_n 是后文要介绍的 Fibonacci 数。

利用 $\gcd$ 还可以求出两个整数 a 和 b 的最小公倍数 $\text{lcm}(a, b)$ 。这个结论很容易由唯一分解定理得到。设

$$\begin{aligned} a &= p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r} \\ b &= p_1^{f_1} p_2^{f_2} \cdots p_r^{f_r} \end{aligned}$$

则

$$\begin{aligned} \gcd(a, b) &= p_1^{\min\{e_1, f_1\}} p_2^{\min\{e_2, f_2\}} \cdots p_r^{\min\{e_r, f_r\}} \\ \text{lcm}(a, b) &= p_1^{\max\{e_1, f_1\}} p_2^{\max\{e_2, f_2\}} \cdots p_r^{\max\{e_r, f_r\}} \end{aligned}$$

由此不难验证 $\gcd(a,b) * \text{lcm}(a,b) = a * b$ 。不过即使有了公式也不要大意。如果把 lcm 写成 $a * b / \gcd(a,b)$, 可能会因此丢掉不少分数—— $a * b$ 可能会溢出! 正确的写法是先除后乘, 即 $a / \gcd(a,b) * b$ 。这样一来, 只要题面上保证最终结果在 int 范围之内, 这个函数就不会出错。但前一份代码却不是这样: 即使最终答案在 int 范围之内, 也有可能中间过程越界。注意这样的细节, 毕竟算法竞赛不是数学竞赛。

10.1.2 Eratosthenes 筛法

无平方因子的数。给出正整数 n 和 m , 区间 $[n, m]$ 内的“无平方因子”的数有多少个? 整数 p 无平方因子, 当且仅当不存在 $k > 1$, 使得 p 是 k^2 的倍数。 $1 \leq n \leq m \leq 10^{12}$, $m - n \leq 10^7$ 。

【分析】

对于这样的限制, 直接枚举判断会超时: 需要判断 10^7 个整数, 而每个整数还需要花费一定的时间判断是否没有平方因子。怎么办呢? 在介绍具体算法之前, 需要学会用 Eratosthenes 筛法构造 $1 \sim n$ 的素数表。

筛法的思想特别简单: 对于不超过 n 的每个非负整数 p , 删除 $2p, 3p, 4p, \dots$, 当处理完所有数之后, 还没有被删除的就是素数。如果用 $\text{vis}[i]$ 表示 i 已经被删除, 筛法的代码可以写成:

```
memset(vis, 0, sizeof(vis));
for(int i = 2; i <= n; i++)
    for(int j = i*2; j <= n; j+=i) vis[j] = 1;
```

尽管可以继续改进, 但这份代码已经相当高效了。为什么呢? 给定外层循环变量 i , 内层循环的次数是 $\left\lfloor \frac{n}{i} \right\rfloor - 1 < \frac{n}{i}$ 。这样, 循环的总次数小于 $\frac{n}{2} + \frac{n}{3} + \dots + \frac{n}{n} = O(n \log n)$ 。这个结论来源于欧拉在 1734 年得到的结果: $1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} = \ln(n+1) + \gamma$, 其中欧拉常数 $\gamma \approx 0.577218$ 。

这样低的时间复杂度允许在很短的时间内得到 10^6 以内的所有素数。

下面来改进这份代码。首先, 在“对于不超过 n 的每个非负整数 p ”中, p 可以限定为素数——只需在第二重循环前加一个判断 $\text{if}(!\text{vis}[i])$ 即可。另外, 内层循环也不必从 $i*2$ 开始——它已经在 $i=2$ 时被筛掉了。改进后的代码如下:

```
int m = sqrt(n+0.5);
memset(vis, 0, sizeof(vis));
for(int i = 2; i <= m; i++) if(!vis[i])
    for(int j = i*i; j <= n; j+=i) vis[j] = 1;
```

这里有一个有意思的问题: 给定的 n , c 的值是多少呢? 换句话说, 不超过 n 的正整数中, 有多少个是素数呢?

素数定理: $\pi(x) \sim \frac{x}{\ln x}$ 。

其中, $\pi(x)$ 表示不超过 x 的素数的个数。上述定理的直观含义是: 它和 $x/\ln x$ 比较接

近——对于算法入门来说，这已足够。表 10-1 给出了一些值来加深读者的印象。

表 10-1 素数定理的直观验证

N	10^2	10^3	10^4	10^5	10^6	10^7	10^8
$\pi(n)$	25	168	1229	9592	78498	664579	5761455
$n/\ln n$	22	145	1086	8686	72382	620421	5428681

最后回到原題：如何求出区间内无平方因子的数？方法和筛素数是类似的：对于不超过 $\sqrt{m}$ 的所有素数 p ，筛掉区间 $[n, m]$ 内 p^2 的所有倍数。

10.1.3 扩展欧几里德算法

直线上的点。求直线 $ax+by+c=0$ 上有多少个整点 (x,y) 满足 $x \in [x_1, x_2]$, $y \in [y_1, y_2]$ 。

【分析】

在解决这个问题之前，首先学习扩展欧几里德算法——找出一对整数 (x,y) ，使得 $ax+by=\gcd(a,b)$ 。注意，这里的 x 和 y 不一定是正数，也可能是负数或者 0。例如， $\gcd(6,15)=3$ ， $6*3-15*1=3$ ，其中 $x=3$ ， $y=-1$ 。这个方程还有其他解，如 $x=-2$ ， $y=1$ 。

下面是扩展欧几里德算法的程序：

```
void gcd(int a, int b, int& d, int& x, int& y) {
    if(!b){ d = a; x = 1; y = 0; }
    else{ gcd(b, a%b, d, y, x); y -= x*(a/b); }
}
```

用数学归纳法并不难证明算法的正确性，此处略去。注意在递归调用时， x 和 y 的顺序变了，而边界也是不难得出的： $\gcd(a,0)=1*a-0*0=a$ 。这样，唯一需要记忆的是 $y=x*(a/b)$ ，哪怕暂时不懂得其中的原因也不要紧。

上面求出了 $ax+by=\gcd(a,b)$ 的一组解 (x_1,y_1) ，那么其他解呢？任取另外一组解 (x_2,y_2) ，则 $ax_1+by_1=ax_2+by_2$ （它们都等于 $\gcd(a,b)$ ），变形得 $a(x_1-x_2)=b(y_2-y_1)$ 。假设 $\gcd(a,b)=g$ ，方程左右两边同时除以 g ^①，得 $a'(x_1-x_2)=b'(y_2-y_1)$ ，其中 $a'=a/g$ ， $b'=b/g$ 。注意，此时 a' 和 b' 互素，因此 x_1-x_2 一定是 b' 的整数倍。设它为 kb' ，计算得 $y_2-y_1=ka'$ 。注意，上面的推导过程并没有用到“ $ax+by$ 的右边是什么”，因此得出如下结论。

提示 10-1： 设 a, b, c 为任意整数。若方程 $ax+by=c$ 的一组整数解为 (x_0,y_0) ，则它的任意整数解都可以写成 (x_0+kb', y_0-ka') ，其中 $a'=a/\gcd(a,b)$ ， $b'=b/\gcd(a,b)$ ， k 取任意整数。

有了这个结论，移项得 $ax+by=-c$ ，然后求出一组解即可。例如：

例 1： $6x+15y=9$ 。根据欧几里德算法，已经得到了 $6 \times (-2) + 15 \times 1 = 3$ ，两边同时乘以 3 得 $6 \times (-6) + 15 \times 3 = 9$ ，即 $x=-6$ ， $y=3$ 时 $6x+15y=9$ 。

例 2： $6x+15=8$ ，两边除以 3 得 $2x+5=8/3$ 。左边是整数，右边不是整数，显然无解。综合起来，有下面的结论。

^① 如果 $g=0$ ，意味着 a 或 b 等于 0，可以特殊判断。

提示 10-2: 设 a, b, c 为任意整数, $g=\gcd(a,b)$, 方程 $ax+by=g$ 的一组解是 (x_0, y_0) , 则当 c 是 g 的倍数时 $ax+by=c$ 的一组解是 $(x_0c/g, y_0c/g)$; 当 c 不是 g 的倍数时无整数解。

这样, 即完整地解决了本问题。顺便说一句, 本题的名称为什么叫“直线上的点”呢? 这是因为在平面坐标系下, $ax+by+c=0$ 是一条直线的方程。

10.1.4 同余与模算术

你需要花多少时间做下面这道题目呢?

$123456789 * 987654321 = (\quad)$

A. 121932631112635266 B. 121932631112635267

C. 121932631112635268 D. 121932631112635269

既然是选择题, 不必费力把答案完整地计算出来——4 个选项的个位数都不相同, 因此只需要计算出答案的最后一位即可。不难得出, 它等于 $1*9=9$ 。把刚才的解题过程抽象出来就是下面的式子:

$$123456789 * 987654321 \bmod 10 = ((123456789 \bmod 10) * (987654321 \bmod 10)) \bmod 10$$

其中 $a \bmod b$ 表示 a 除以 b 的余数, C 语言表达式是 $a \% b$ 。在本章中, b 一定是正整数, 尽管 $b < 0$ 时表达式 $a \% b$ 也是合法的 (但 $b = 0$ 时会出现除零错)。

不难得到下面的公式:

$$(a + b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$$

$$(a - b) \bmod n = ((a \bmod n) - (b \bmod n) + n) \bmod n$$

$$ab \bmod n = (a \bmod n)(b \bmod n) \bmod n$$

注意在减法中, 由于 $a \bmod n$ 可能小于 $b \bmod n$, 需要在结果加上 n , 而在乘法中, 需要注意 $a \bmod n$ 和 $b \bmod n$ 相乘是否会溢出。例如, 当 $n=10^9$ 时, $ab \bmod n$ 一定在 int 范围内, 但 $a \bmod n$ 和 $b \bmod n$ 的乘积可能会超过 int 。需要用 long long 保存中间结果, 例如:

```
int mul_mod(int a, int b, int n) {
    a %= n; b %= n;
    return (int)((long long)a * b % n);
}
```

当然, 如果 n 本身超过 int 但又在 long long 范围内, 上述方法就不适用了。在这种情况下, 建议初学者使用高精度乘法——尽管有办法可以避免, 但技巧性很强, 不推荐初学者学习。

大整数取模。输入正整数 n 和 m , 输出 $n \bmod m$ 的值。 $n \leq 10^{100}$, $m \leq 10^9$ 。

【分析】

首先, 把大整数写成“自左向右”的形式: $1234 = ((1*10+2)*10+3)*10+4$, 然后用前面的公式, 每步取模, 例如:

```
scanf("%s%d", n, &m);
int len = strlen(n);
int ans = 0;
```

```
for(int i = 0; i < len; i++)
    ans = (int)((long long)ans*10 + n[i] - '0') % m;
printf("%d\n", ans);
```

当然，也可以把 `ans` 声明成 `long long` 类型的，然后在输出时临时转换为 `int`，但要注意乘法溢出的问题。

幂取模。输入正整数 a 、 n 和 m ，输出 $a^n \bmod m$ 的值。 $a, n, m \leq 10^9$ 。

【分析】

很容易写出下面的代码：

```
int pow_mod(int a, int n, int m) {
    int ans = 1;
    for(int i = 0; i < n; i++) ans = (int)((long long)ans * a % m);
}
```

这个函数的时间复杂度为 $O(n)$ ，当 n 很大时速度很不理想。有没有办法算得更快呢？可以利用分治法：

```
int pow_mod(int a, int n, int m) {
    if(n == 0) return 1;
    int x = pow_mod(a, n/2, m);
    long long ans = (long long)x * x % m;
    if(n%2 == 1) ans = ans * a % m;
    return (int)ans;
}
```

例如， $a^{29} = (a^{14})^2 * a$ ，而 $a^{14} = (a^7)^2$ ， $a^7 = (a^3)^2 * a$ ， $a^3 = a^2 * a$ ，一共只做了 7 次乘法。不知读者有没有发现，上述递归方式和二分查找很类似——每次规模近似减小一半。因此，时间复杂度为 $O(\log n)$ ，比 $O(n)$ 好了很多。

模线性方程组。输入正整数 a, b, n ，解方程 $ax \equiv b \pmod{n}$ 。 $a, b, n \leq 10^9$ 。

【分析】

本题中出现了一个新记号：同余。 $a \equiv b \pmod{n}$ 的含义是“ a 和 b 关于模 n 同余”，即 $a \bmod n = b \bmod n$ 。不难得出， $a \equiv b \pmod{n}$ 的充要条件是： $a-b$ 是 n 的整数倍。

提示 10-3： $a \equiv b \pmod{n}$ 的含义是“ a 和 b 除以 n 的余数相同”，其充要条件是“ $a-b$ 是 n 的整数倍”。

这样，原来的方程就可以理解成： $ax-b$ 是 n 的正整数倍。设这个“倍数”为 y ，则 $ax-b=ny$ ，移项得 $ax-ny=b$ ，这恰好就是 10.1.3 节介绍的不定方程（ a, n, b 是已知量， x 和 y 是未知数）！接下来的步骤不再介绍。唯一需要说明的是，如果 x 是方程的解，满足 $x \equiv y \pmod{n}$ 的其他整数 y 也是方程的解。因此，当谈到同余方程的一个解时，其实指的是一个同余等价类。

尽管算法已无须继续讨论，有一个特殊情况需要引起读者重视。 $b=1$ 时， $ax \equiv 1 \pmod{n}$ 的解称为 a 关于模 n 的逆（inverse），它类似于实数运算中“倒数”的概念。什么时候 a 的逆存在呢？根据上面的讨论，方程 $ax-ny=1$ 要有解。这样，1 必须是 $\gcd(a, n)$ 的倍数，因

此 a 和 n 必须互素（即 $\gcd(a, n) = 1$ ）。在满足这个条件的前提下， $ax \equiv 1 \pmod{n}$ 只有唯一解。注意，同余方程的解是指一个等价类。

提示 10-4: 方程 $ax \equiv 1 \pmod{n}$ 的解称为 a 关于模 n 的逆。当 $\gcd(a, n) = 1$ 时，该方程有唯一解；否则，该方程无解。

10.1.5 应用举例

例题 10-1 巨大的斐波那契数！（Colossal Fibonacci Numbers!, UVa11582）

输入两个非负整数 a 、 b 和正整数 n ($0 \leq a, b < 2^{64}$, $1 \leq n \leq 1000$)，你的任务是计算 $f(a^b)$ 除以 n 的余数。其中 $f(0) = f(1) = 1$ ，且对于所有非负整数 i ， $f(i+2) = f(i+1) + f(i)$ 。

【分析】

所有计算都是对 n 取模的，不妨设 $F(i) = f(i) \bmod n$ 。不难发现，当二元组 $(F(i), F(i+1))$ 出现重复时，整个序列就开始重复。例如， $n=3$ ，序列 $F(i)$ 的前 10 项为 1, 1, 2, 0, 2, 2, 1, 0, 1, 1，第 9、10 项和前两项完全一样。根据递推公式，第 11 项会等于第 3 项，第 12 项等于第 4 项……

多久会出现重复呢？因为余数最多 n 种，所以最多 n^2 项就会出现重复。设周期为 M ，则只需计算出 $F(0) \sim F(n^2)$ ，然后算出 $F(a^b)$ 等于其中的哪一项即可。

例题 10-2 不爽的裁判（Disgruntled Judge, NWERC 2008, UVa12169）

有个裁判出的题太难，总是没人做，所以他很不爽。有一次他终于忍不住了，心想：“反正我的题没人做，我干嘛要费那么多心思出题？不如就输入一个随机数，输出一个随机数吧。”

于是他找了 3 个整数 x_1 、 a 和 b ，然后按照递推公式 $x_i = (ax_{i-1} + b) \bmod 10001$ 计算出了一个长度为 $2T$ 的数列，其中 T 是测试数据的组数。然后，他把 T 和 $x_1, x_3, \dots, x_{2T-1}$ 写到输入文件中， $x_2, x_4, \dots, x_{2T}$ 写到了输出文件中。

你的任务就是解决这个疯狂的题目：输入 $T, x_1, x_3, \dots, x_{2T-1}$ ，输出 $x_2, x_4, \dots, x_{2T}$ 。输入保证 $T \leq 100$ ，且输入的所有 x 值为 $0 \sim 10000$ 的整数。如果有多种可能的输出，任意输出一个即可。

【分析】

如果知道了 a ，就可以计算出 x_2 ，进而根据 $x_3 = (ax_2 + b) \bmod 10001$ 算出 b 。有了 x_1 、 a 和 b ，就可以在 $O(T)$ 时间内计算出整个序列了。如果在计算过程中发现和输入矛盾，则这个 a 是非法的。由于 a 是 $0 \sim 10000$ 的整数（因为递推公式对 10001 取模），即使枚举所有的 a ，时间效率也足够高。

例题 10-3 选择与除法（Choose and Divide, UVa10375）

已知 $C(m, n) = m! / (n!(m-n)!)$ ，输入整数 p, q, r, s ($p \geq q, r \geq s, p, q, r, s \leq 10000$)，计算 $C(p, q) / C(r, s)$ 。输出保证不超过 10^8 ，保留 5 位小数。

【分析】

本题正是唯一分解定理的用武之地。组合数 $C(m, n)$ 的性质将在 10.2.1 节中介绍，本题只需要用到它的定义。

首先, 求出 10000 以内的所有素数 `primes`, 然后用数组 `e` 表示当前结果的唯一分解式中各个素数的指数。例如, $e=\{1,0,2,0,0,\dots\}$ 表示 $2^1 \cdot 5^2 = 50$ 。主程序如下:

```
while(cin >> p >> q >> r >> s) {
    memset(e, 0, sizeof(e));
    add_factorial(p, 1);
    add_factorial(q, -1);
    add_factorial(p-q, -1);
    add_factorial(r, -1);
    add_factorial(s, 1);
    add_factorial(r-s, 1);
    double ans = 1;
    for(int i = 0; i < primes.size(); i++)
        ans *= pow(primes[i], e[i]);
    printf("%.5lf\n", ans);
}
```

其中 `add_factorial(n,d)` 表示把结果乘以 $(n!)^d$, 它的实现如下:

//乘以或除以 n . $d=0$ 表示乘, $d=-1$ 表示除

```
void add_integer(int n, int d) {
    for(int i = 0; i < primes.size(); i++) {
        while(n % primes[i] == 0) {
            n /= primes[i];
            e[i] += d;
        }
        if(n == 1) break; //提前终止循环, 节约时间
    }
}
```

```
void add_factorial(int n, int d) {
    for(int i = 1; i <= n; i++)
        add_integer(i, d);
}
```

例题 10-4 最小公倍数的最小和 (Minimum Sum LCM, UVa10791)

输入整数 n ($1 \leq n < 2^{31}$), 求至少两个正整数, 使得它们的最小公倍数为 n , 且这些整数的和最小。输出最小的和。

【分析】

本题再次用到了唯一分解定理。设唯一分解式 $n = a_1^{p_1} \cdot a_2^{p_2} \dots$, 不难发现每个 $a_i^{p_i}$ 作为一个单独的整数时最优。

如果就这样匆匆编写程序, 可能会掉入陷阱。本题有好几个特殊情况要处理: $n=1$ 时答案为 $1+1=2$; n 只有一种因子时需要加个 1, 还要注意 $n=2^{31}-1$ 时不要溢出。

例题 10-5 GCD 等于 XOR (GCD XOR, ACM/ICPC Dhaka 2013, UVa12716)

输入整数 $n (1 \leq n \leq 300000000)$, 有多少对整数 (a, b) 满足: $1 \leq b \leq a \leq n$, 且 $\gcd(a, b) = a \text{ XOR } b$ 。例如 $n=7$ 时, 有 4 对: $(3, 2), (5, 4), (6, 4), (7, 6)$ 。

【分析】

本题看上去很难找到简洁的数学公式, 因为 $\gcd$ 和 xor 看上去似乎毫不相干。不过 xor 的好处是: $a \text{ xor } b = c$, 则 $a \text{ xor } c = b$, 所以可以枚举 a 和 c , 然后算出 $b = a \text{ xor } c$, 最后验证一下是否有 $\gcd(a, b) = c$ 。时间复杂度如何? 因为 c 是 a 的约数, 所以和素数筛法类似, 时间复杂度为 $n/1 + n/2 + \dots + n/n = O(n \log n)$ 。再加上 $\gcd$ 的时间复杂度为 $O(\log n)$, 所以总的时间复杂度为 $O(n(\log n)^2)$ 。

我们还可以做得更好。上述程序写出来之后, 可以打印一些满足 $\gcd(a, b) = a \text{ xor } b = c$ 的三元组 (a, b, c) , 然后很容易发现一个现象: $c = a - b$ 。

证明如下: 不难发现 $a - b \leq a \text{ xor } b$, 且 $a - b \geq c$ 。假设存在 c 使得 $a - b > c$, 则 $c < a - b \leq a \text{ xor } b$, 与 $c = a \text{ xor } b$ 矛盾。

有了这个结论, 还是沿用上述算法, 枚举 a 和 c , 计算 $b = a - c$, 则 $\gcd(a, b) = \gcd(a, a - c) = c$, 因此只需验证是否有 $c = a \text{ xor } b$, 时间复杂度降为了 $O(n \log n)$ 。

10.2 计数与概率基础

排列与组合是最基本的计数技巧。本节介绍一些基本的相关知识和方法, 供读者参考。

加法原理。做一件事情有 n 个办法, 第 i 个办法有 p_i 种方案, 则一共有 $p_1 + p_2 + \dots + p_n$ 种方案。

乘法原理。做一件事情有 n 个步骤, 第 i 个步骤有 p_i 种方案, 则一共有 $p_1 p_2 \dots p_n$ 种方案。

乘法原理是加法原理的特殊情况 (按照第一步骤进行分类), 二者都可用于递推。注意应用加法原理的关键是分类: 各类别之间必须没有重复、没有遗漏。如果有重复, 可以使用容斥原理。

容斥原理。假设班里有 10 个学生喜欢数学, 15 个学生喜欢语文, 21 个学生喜欢编程, 一共有多少个学生呢? 是 $10 + 15 + 21 = 46$ 个吗? 不是的, 因为有些学生可能同时喜欢数学和语文, 或者语文和编程, 甚至还可能有三者都喜欢的。为了叙述方便, 将喜欢语文、数学、编程的学生集合分别用 A, B, C 表示, 则学生总数等于 $|A \cup B \cup C|$ 。刚才已经说了, 如果把这 3 个集合的元素个数 $|A|, |B|, |C|$ 直接加起来, 会有一些元素重复统计了, 因此需要扣掉 $|A \cap B|, |B \cap C|, |C \cap A|$, 但这样一来, 又有一小部分多扣了, 需要加回来: $|A \cap B \cap C|$ 。这样, 就得到了一个公式:

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |B \cap C| - |C \cap A| + |A \cap B \cap C|$$

一般地, 对于任意多个集合, 都可以列出这样一个等式, 其中左边是所有集合的并的元素个数, 右边是这些集合的“各种搭配”。每个“搭配”都是若干个集合的交集, 且每一项前面的正负号取决于集合的个数——奇数个集合为正, 偶数个集合为负。

有重复元素的全排列。有 k 个元素，其中第 i 个元素有 n_i 个，求全排列个数。

【分析】

令所有 n_i 之和为 n ，再设答案为 x 。首先做全排列，然后把所有元素编号，其中第 s 种元素编号为 $1 \sim n_s$ （例如，有 3 个 a ，两个 b ，先排列成 $aabba$ ，然后可以编号为 $a_1a_3b_2b_1a_2$ ）。这样做以后，由于编号后所有元素均不相同，方案总数为 n 的全排列数 $n!$ 。根据乘法原理，得到了一个方程： $n_1!n_2!n_3!\cdots n_k!x=n!$ ，移项即可。

可重复选择的组合。有 n 个不同元素，每个元素可以选多次，一共选 k 个元素，有多少种方法？例如， $n=3$ ， $k=2$ 时有 6 种： $(1,1), (1,2), (1,3), (2,2), (2,3), (3,3)$ 。

【分析】

设第 i 个元素选 x_i 个，问题转化为求方程 $x_1+x_2+\cdots+x_n=k$ 的非负整数解的个数。令 $y_i=x_i+1$ ，则答案为 $y_1+y_2+\cdots+y_n=k+n$ 的正整数解的个数。想象有 $k+1$ 个数字“1”排成一排，则问题等价于：把这些“1”分成 n 个部分，有多少种方法？这相当于在 $k+n-1$ 个“候选分隔线”中选 $n-1$ 个，即 $C(k+n-1, n-1)=C(n+k-1, k)$ 。

10.2.1 杨辉三角与二项式定理

组合数 C_n^m 在组合数学中占有重要地位。与组合数相关的最重要的两个内容是杨辉三角和二项式定理。如图 10-1 所示就是一个杨辉三角。

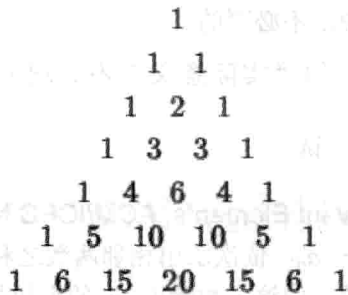


图 10-1 杨辉三角

另一方面，把 $(a+b)^n$ 展开，将得到一个关于 x 的多项式：

$$(a+b)^0=1$$

$$(a+b)^1=a+b$$

$$(a+b)^2=a^2+2ab+b^2$$

$$(a+b)^3=a^3+3a^2b+3ab^2+b^3$$

$$(a+b)^4=a^4+4a^3b+6a^2b^2+4ab^3+b^4$$

系数正好和杨辉三角一致。一般地，有二项式定理：

$$(a+b)^n=\sum_{k=0}^n C_n^k a^{n-k} b^k$$

这不难理解： $(a+b)^n$ 是 n 个括号连乘，每个括号里任选一项乘起来都会对最后的结果

有一个贡献。如果选了 k 个 a , 就一定会选 $n-k$ 个 b , 最后的项自然就是 $a^{n-k}b^k$ 。而从 n 个 a 里选 k 个 (同时也相当于 n 个 b 里选 $n-k$ 个) 有 C_n^k 种方法, 这也是组合数的定义。

给定 n , 如何求出 $(a+b)^n$ 中所有项的系数呢? 一个方法是用递推, 根据杨辉三角中不难发现的规律, 可以写出如下程序:

```
memset(C, 0, sizeof(C));
for(int i = 0; i <= n; i++) {
    C[i][0] = 1;
    for(int j = 1; j <= i; j++) C[i][j] = C[i-1][j-1] + C[i-1][j];
}
```

但遗憾的是, 这个算法的时间复杂度是 $O(n^2)$ ——尽管只用了杨辉三角的第 n 行的 $n+1$ 个元素, 却把全部 n 行的 $O(n^2)$ 个元素都计算了一遍。

另一个方法是利用等式 $C_n^k = \frac{n-k+1}{k} C_n^{k-1}$, 从 $C_n^0 = 1$ 开始从左到右递推, 例如:

```
C[0] = 1;
for(int i = 1; i <= n; i++) C[i] = C[i-1]*(n-i+1)/i;
```

注意, 应该先乘后除, 因为 $C[i-1]/i$ 可能不是整数。但这样一来增加了溢出的可能性——即使最后结果在 `int` 或 `long long` 范围之内, 乘法也可能溢出。如果担心这样的情况出现, 可以先约分, 不过一般来说是不必要的。

尽管等式 $C_n^k = \frac{n-k+1}{k} C_n^{k-1}$ 的“实际意义”不是很明显, 却很容易用组合数公式

$C_n^k = \frac{n!}{k!(n-k)!}$ 证明, 读者不妨一试。

例题 10-6 无关的元素 (Irrelevant Elements, ACM/ICPC NEERC 2004, UVa1635)

对于给定的 n 个数 $a_1, a_2, \dots, a_n$, 依次求出相邻两数之和, 将得到一个新数列。重复上述操作, 最后结果将变成一个数。问这个数除以 m 的余数与哪些数无关? 例如 $n=3, m=2$ 时, 第一次求和得到 a_1+a_2, a_2+a_3 , 再求和得到 $a_1+2a_2+a_3$, 它除以 2 的余数和 a_2 无关。 $1 \leq n \leq 10^5, 2 \leq m \leq 10^9$ 。

【分析】

显然最后的求和式是 $a_1, a_2, \dots, a_n$ 的线性组合。设 a_i 的系数为 $f(i)$, 则和式除以 m 的余数与 a_i 无关, 当且仅当 $f(i)$ 是 i 的倍数。不妨看一个简单的例子:

$$\begin{array}{ccccccccc}
 & a_1 & & a_2 & & a_3 & & a_4 & & a_5 \\
 a_1 + a_2 & & a_2 + a_3 & & a_3 + a_4 & & a_4 + a_5 & & & \\
 a_1 + 2a_2 + a_3 & & a_2 + 2a_3 + a_4 & & a_3 + 2a_4 + a_5 & & & & & \\
 a_1 + 3a_2 + 3a_3 + a_4 & & a_2 + 3a_3 + 3a_4 + a_5 & & & & & & & \\
 a_1 + 4a_2 + 6a_3 + 4a_4 + a_5 & & & & & & & & &
 \end{array}$$

看到最后的结果, 你想到了什么? 没错, “1 4 6 4 1”正是杨辉三角的第 5 行! 不难证明, 在一般情况下, 最后 a_i 的系数是 C_{n-1}^{i-1} 。这样, 问题就变成了 $C_{n-1}^0, C_{n-1}^1, \dots, C_{n-1}^{n-1}$ 中有

哪些是 m 的倍数。

还记得二项式展开的方法吗? 理论上, 利用此方法可以递推出所有 C_{n-1}^{i-1} , 但它们太大了, 必须用高精度才能存得下。但此问题中所关心的只是“哪些是 m 的倍数”, 受到数论部分中的启发, 只需要依次计算 m 的唯一分解式中各个素因子在 C_{n-1}^{i-1} 中的指数即可完成判断。这些指数仍然可以用 $C_n^k = \frac{n-k+1}{k} C_n^{k-1}$ 递推, 并且不会涉及高精度。有的读者可能会尝试直接递推每个系数除以 m 的余数, 但遗憾的是, 递推式中有除法, 而模 m 意义下的逆并不一定存在。

10.2.2 数论中的计数问题

约数的个数。 给出正整数 n 的唯一分解式 $n = p_1^{a_1} p_2^{a_2} p_3^{a_3} \cdots p_k^{a_k}$, 求 n 的正约数的个数。

【分析】

不难看出, n 的任意正约数也只能包含 p_1, p_2, p_3 等素因子, 而不能有新的素因子出现。对于 n 的某个素因子 p_i , 它在所求约数中的指数可以是 $0, 1, 2, \cdots, a_i$ 共 a_i+1 种情况, 而且不同的素因子之间相互独立。根据乘法原理, n 的正约数个数为:

$$\prod_{i=1}^k (a_i + 1) = (a_1 + 1)(a_2 + 1) \cdots (a_k + 1)$$

小于 n 且与 n 互素的整数个数。 给出正整数 n 的唯一分解式 $n = p_1^{a_1} p_2^{a_2} p_3^{a_3} \cdots p_k^{a_k}$, 求 $1, 2, 3, \cdots, n$ 中与 n 互素的数的个数。

【分析】

用容斥原理。首先从总数 n 中分别减去是 $p_1, p_2, \cdots, p_k$ 的倍数的个数 (对于素数 p 来说, “与 p 互素” 和 “不是 p 的倍数” 等价), 即 $n - \frac{n}{p_1} - \frac{n}{p_2} - \cdots - \frac{n}{p_k}$, 然后加上 “同时是两个素因子的倍数” 的个数 $\frac{n}{p_1 p_2} + \frac{n}{p_1 p_3} + \cdots + \frac{n}{p_{k-1} p_k}$, 再减去 “同时是 3 个素因子的倍数” —— 写成一个 “学术味比较浓” 的公式就是:

$$\varphi(n) = \sum_{S \subseteq \{p_1, p_2, \dots, p_k\}} (-1)^{|S|} \prod_{p_i \in S} \frac{n}{p_i}$$

这里引入的新记号 $\varphi(n)$ 就是题目中所求的结果, 称为欧拉函数。强烈建议初学者花一些时间理解这个公式。对于 $\{p_1, p_2, \dots, p_k\}$ 的任意子集 S , “不与其中任何一个互素” 的元素个数是 $\frac{n}{\prod_{p_i \in S} p_i}$ 。不过这一项的前面是加号还是减号呢? 这取决于 S 中的元素个数——奇数

个就是 “减号”, 偶数个就是 “加号”。

公式已得出, 可计算起来很不方便。如果直接根据公式, 需要计算多达 2^k 项的代数和, 甚至可能比 “暴力枚举 (依次判断 $1 \sim n$ 中每个数是否与 n 互素)” 还要慢。

下一步并不显然。上述公式可以变形成如下的形式:

$$\varphi(n) = n \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_k}\right)$$

从而只需要 $O(k)$ 的计算时间, 在刚才的基础上大大提高了效率。为什么这个式子和上一个等价呢? 直接考虑新公式的“展开方式”即可。展开式的每一项是从每个括号各选一个 (选 1 或者 $-\frac{1}{p_i}$), 全部乘起来以后再乘以 n 得到。这不正是最初的推导过程吗?

如果没有给出唯一分解式, 需要用试除法依次判断 $\sqrt{n}$ 内的所有素数是否是 n 的因子。这样, 则需要先生成 $\sqrt{n}$ 内的素数表。但其实并不用这么麻烦: 只需要每次找到一个素因子之后把它“除干净”, 即可保证找到的因子都是素数 (想一想, 为什么)。

```
int euler_phi(int n) {
    int m = (int)sqrt(n+0.5);
    int ans = n;
    for(int i = 2; i <= m; i++) if(n % i == 0) {
        ans = ans / i * (i-1);
        while(n % i == 0) n /= i;
    }
    if(n > 1) ans = ans / n * (n-1);
}
```

1~ n 中所有数的欧拉 ϕ 函数值。并不需要依次计算。可以用与筛法求素数非常类似的方法, 在 $O(n \log \log n)$ 时间内计算完毕, 例如 (原理请读者体会):

```
void phi_table(int n, int* phi) {
    for(int i = 2; i <= n; i++) phi[i] = 0;
    phi[1] = 1;
    for(int i = 2; i <= n; i++) if(!phi[i])
        for(int j = i; j <= n; j += i) {
            if(!phi[j]) phi[j] = j;
            phi[j] = phi[j] / i * (i-1);
        }
}
```

例题 10-7 交表 (Send a Table, UVa10820)

有一道比赛题目, 输入两个整数 x, y ($1 \leq x, y \leq n$), 输出某个函数 $f(x, y)$ 。有位选手想交表 (即事先计算出所有的 $f(x, y)$, 写在源代码里), 但是表太大了, 源代码超过了比赛的限制, 需要精简。

好在那道题目有一个性质, 使得很容易根据 $f(x, y)$ 算出 $f(x*k, y*k)$ (其中 k 是任意正整数), 这样有一些 $f(x, y)$ 就不需要存在表里了。

输入 n ($n \leq 50000$), 你的任务是统计最简的表里有多少个元素。例如, $n=2$ 时有 3 个: $(1, 1), (1, 2), (2, 1)$ 。

【分析】

本题的本质是: 输入 n , 有多少个二元组 (x, y) 满足: $1 \leq x, y \leq n$, 且 x 和 y 互素。不难发现除了 $(1, 1)$ 之外, 其他二元组 (x, y) 中的 x 和 y 都不相等。设满足 $x < y$ 的二元组有 $f(n)$ 个, 那

么答案就是 $2f(n)+1$ 。

对照欧拉函数的定义,可以得到 $f(n)=\phi(2)+\phi(3)+\cdots+\phi(n)$, 时间复杂度为 $O(n\log\log n)$ 。

10.2.3 编码与解码

两个 a 、一个 b 和一个 c 组成的所有串可以按照字典序编号为:

$aabc(1)$ 、 $aacb(2)$ 、 $abac(3)$ 、 $\cdots$ 、 $cbaa(12)$

任给一个字符串, 能否方便地求出它的编号呢? 例如, 输入 $acab$, 则应输出 5。

下面直接求解一般情况的问题(并不限定字母的种类和个数)。设输入串为 S , 记 $d(S)$ 为 S 的各个排列中, 字典序比 S 小的串的个数, 则可以用递推法求解 $d(S)$, 如图 10-2 所示。

其中边上的字母表示“下一个字母”, $f(x)$ 表示多重集 x 的全排列个数。例如, 根据第一个字母, 可以把字典序小于 $caba$ 的字符串分为 3 种: 以 a 开头的, 以 b 开头的, 以 c 开头的, 分别对应 $d(caba)$ 的 3 棵子树。以 a 开头的所有串的字典序都小于 $caba$, 所以剩下的字符可以任意排列, 个数为 $f(cba)$; 同理, 以 b 开头的所有串的字典序也都小于 $caba$, 个数为 $f(caa)$; 以 c 开头的串字典序不一定小于 $caba$, 关键要看后 3 个字符, 因此这部分的个数为 $d(aba)$, 还需要继续往下分。

至于 f 函数的求解, 大部分组合数学书籍中均有介绍: 设字符一共有 k 类, 个数分别为 $n_1, n_2, \cdots, n_k$, 则这个多重集的全排列个数为 $\frac{(n_1 + n_2 + \cdots + n_k)!}{n_1! n_2! \cdots n_k!}$ 。

不难算出, $f(caa) = \frac{(1+2)!}{1!2!} = \frac{6}{2} = 3$, 其他 f 值分别为 $f(cba)=6, f(b)=1$, 故 $d(caba)=f(cba)+f(caa)+f(b)=3+6+1=10$ 。既然“比它小”的个数是 10, 序号自然就是 11 了。

“给物体一个编号”称为编码, 同理也有“解码”, 即根据序号构造出这个物体。这个过程和刚才的很接近: 依次确定各个位置上的字母即可。例如, 要求出序号为 8 (因此有 7 个比它小) 的字符串, 推理过程如图 10-3 所示。

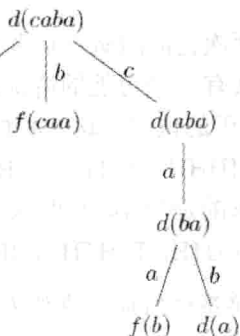


图 10-2 字符串编码的递推过程

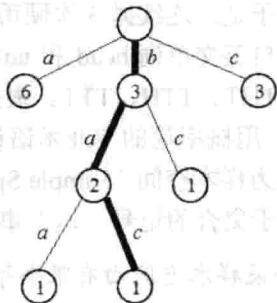


图 10-3 字符串解码的递推过程

例题 10-8 密码 (Password, ACM/ICPC Daejon 2010, UVa1262)

给两个 6 行 5 列的字母矩阵, 找出满足如下条件的“密码”: 密码中的每个字母在两个矩阵的对应列中均出现。例如, 左数第 2 个字母必须在两个矩阵中的左数第 2 列中均出现。例如, 图 10-4 中, COMPU 和 DPMAG 都满足条件。

A	Y	G	S	U
D	O	M	R	A
C	P	F	A	S
X	B	O	D	G
W	D	Y	P	K
P	R	X	W	O

C	B	O	P	T
D	O	S	B	G
G	T	R	A	R
A	P	M	M	S
W	S	X	N	U
E	F	G	H	I

图 10-4 满足条件的密码

字典序最小的5个满足条件的密码分别是：ABGAG、ABGAS、ABGAU、ABGPG和ABGPS。给定 k ($1 \leq k \leq 7777$)，你的任务是找出字典序第 k 小的密码。如果不存在，输出 NO。

【分析】

本题是一个经典的解码问题。首先把不可能出现在答案中的字母排除。例如在上面的例子中，第1个字母只能是{A,C,D,W}，第2个字母只能是{B,O,P}，第3个字母只能是{G,M,O,X}，第4个字母只能是{A,P}，第5个字母只能是{G,S,U}。

不管第1个字母是多少，后4个字母都有 $3 \times 4 \times 2 \times 3 = 72$ 种可能，因此当 $k \leq 72$ 时，第1个字母是A，当 $72 < k \leq 144$ 时第1个字母是C，如此等等。再用同样的方法确定第2，3，4，5个字母即可。

由于 $k \leq 7777$ ，本题还有一个取巧的方法：直接按照字典序从小到大的顺序递归一个一个的枚举。虽然代码比递推法要长，但是由于思维难度小，往往能在更短的时间内写完、写对。

10.2.4 离散概率初步

关于概率有一套很深的理论，不过很多和概率相关的问题并不需要特别的知识，熟悉排列组合就够了。

第1个例子是：连续抛3次硬币，恰好有两次正面的概率是多少？用H和T来表示正面和背面（取自英文单词 head 和 tail），则一共有8种可能的情况：HHH、HHT、HTH、HTT、THH、THT、TTH、TTT。根据我们对硬币的认识，这8种情况出现的可能性相同，概率各为 $1/8$ 。用概率论的专业术语说，这里的{HHH、HHT、HTH、HTT、THH、THT、TTH、TTT}称为样本空间（Sample Space）。所求的是“恰好有两次正面”这个事件（Event）的概率。借助于集合的记号，这个事件可以表示为{HHT, HTH, THH}，其概率为 $3/8$ 。

提示 10-5：如果样本空间由有限个等概率的简单事件组成，事件 E 的概率可以用组合计数的方法得到： $P(E) = \frac{|E|}{|S|}$ 。

第2个例子是：如果一间屋子里有23个人，那么“至少有两个人的生日相同”的概率超过50%。为了简单起见，假定已知每个人的生日都不是2月29日。

尽管看上去复杂了许多，其实这个例子和抛硬币是类似的。每个人的生日是365天中

等概率随机选择的, 因此样本空间大小 $|S| = 365^{23}$ 。接下来需要计算“至少有两个人生日相同”的情况有多少种。这个数目不太好直接统计, 所以统计“任何两个人的生日都不相同”的数目, 然后用总数减去它即可。公式不难得到:

$$P(E) = \frac{|E|}{|S|} = \frac{|S| - |\bar{E}|}{|S|} = 1 - \frac{P_{365}^{23}}{365^{23}}$$

不管是 P_{365}^{23} 还是 365^{23} 都无法储存在 int 或者 long long 中, 但概率是实数, 并且此处并不需要太高的精度, 所以可以直接计算, 例如:

```
double P(int n, int m) {
    double ans = 1.0;
    for(int i = 0; i < m; i++) ans *= (n-i);
    return ans;
}
```

```
double birthday(int n, int m) {
    double ans = P(n, m);
    for(int i = 0; i < m; i++) ans /= n;
    return 1 - ans;
}
```

函数 birthday(365,23)的返回值为 0.5073, 即 50.73%。别高兴得太早, 我们来算一算 birthday(365,365)。直观上, 365 个人中几乎肯定会有两个人的生日相同, 因此 birthday(365,365)应该返回一个很接近 1 的值。可结果呢? 很不幸, 返回值为 -1.#INF0000——连 double 都溢出了。

解决方案是边乘边除, 而不是连着乘 m 次, 然后再连着除 m 次。例如:

```
double birthday(int n, int m) {
    double ans = 1.0;
    for(int i = 0; i < m; i++) ans *= (double)(n-i) / n;
    return 1 - ans;
}
```

本例说明: 正如数论和组合计数中要注意 int 和 long long 溢出一样, 在概率计算中要注意 double 溢出。顺便说一句, 这个“改进版”程序其实有个直接的概率意义:

$$P(E) = 1 - P(\bar{E}) = 1 - P(E_1)P(E_2)P(E_3) \cdots P(E_m) = 1 - \frac{n}{n} \times \frac{n-1}{n} \times \frac{n-2}{n} \cdots \frac{n-(m-1)}{n}$$

其中, E_i 表示“第 i 个人的生日不和前面的人重复”这个事件。上面的公式用到了这样一个结论: 如果有 n 个相互独立的事件, 则它们同时发生的概率是每个事件单独发生的概率的乘积, 像计数中的乘法原理一样。看上去很直观吧? 但严格的定义需要用到“条件概率”的知识。

条件概率。在概率计算中, 条件概率扮演了重要的作用。公式如下:

$$P(A|B) = P(AB) / P(B)$$

这里， $P(A|B)$ 是指，在事件 B 发生的前提下，事件 A 发生的概率，而 $P(AB)$ 是指两个事件 A 和 B 同时发生的概率。前面所说的两个事件 AB 独立就是指 $P(AB)=P(A)P(B)$ 。

条件概率中还有一个重要的公式，即贝叶斯公式： $P(A|B)=P(B|A) * P(A)/P(B)$

全概率公式。计算概率的一种常用方法是：样本空间 S 分成若干个不相交的部分 $B_1, B_2, \dots, B_n$ ，则 $P(A)=P(A|B_1)*P(B_1)+P(A|B_2)*P(B_2)+\dots+P(A|B_n)*P(B_n)$ 。

公式看上去复杂，但其实思路很简单。例如，参加比赛，得一等奖、二等奖、三等奖和优胜奖的概率分别为 0.1、0.2、0.3 和 0.4，这 4 种情况下，你会被妈妈表扬的概率分别为 1.0、0.8、0.5、0.1，则你被妈妈表扬的总概率为 $0.1*1.0+0.2*0.8+0.3*0.5+0.4*0.1=0.45$ 。使用全概率公式的关键是“划分样本空间”，只有把所有可能情况不重复、不遗漏地进行分类，并算出每个分类下事件发生的概率，才能得出该事件发生的总概率。

例题 10-9 决斗 (Headshot, ACM/ICPC NEERC 2009, UVa1636)

首先在手枪里随机装一些子弹，然后抠了一枪，发现没有子弹。你希望下一枪也没有子弹，是应该直接再抠一枪（输出 SHOOT）呢，还是随机转一下再抠（输出 ROTATE）？如果两种策略下没有子弹的概率相等，输出 EQUAL。

手枪里的子弹可以看成是一个环形序列，开枪一次以后对准下一个位置。例如，子弹序列为 0011 时，第一次开枪前一定在位置 1 或 2（因为第一枪没有子弹），因此开枪之后位于位置 2 或 3。如果此时开枪，有一半的概率没有子弹。序列长度为 2~100。

【分析】

直接抠一枪没子弹的概率是一个条件概率，等于子串 00 的个数除以 00 和 01 总数（也就是 0 的个数）。转一下再抠没子弹的概率等于 0 的比率。

设子串 00 的个数为 a ，0 的个数为 b ，则两个概率分别是 a/b 和 b/n 。问题就是比较 an 和 b^2 。前者大就是 SHOOT，后者大就是 ROTATE。

例题 10-10 奶牛和轿车 (Cows and Cars, UVa10491)

有这么一个电视节目：你的面前有 3 个门，其中两扇门里是奶牛，另外一扇门里则藏着奖品——一辆豪华小轿车。在你选择一扇门之后，门并不会立即打开。这时，主持人会给你个提示，具体方法是打开其中一扇有奶牛的门（不会打开你已经选择的那个门，即使里面是牛）。接下来你有两种可能的决策：保持先前的选择，或者换成另外一扇未开的门。当然，你最终选择打开的那扇门后面的东西就归你了。

在这个例子里面，你能得到轿车的概率是 $2/3$ （难以置信吧！），方法是总是改变自己的选择。 $2/3$ 这个数是这样得到的：如果选择了两个牛之一，你肯定能换到车前面的门，因为主持人已经让你看了另外一个牛；而如果你开始选择的的就是车，就会换成剩下的牛并且输掉奖品。由于你的最初选择是任意的，因此选错的概率是 $2/3$ 。也正是这 $2/3$ 的情况让你能换到那辆车（另外 $1/3$ 的情况你会从车切换到牛）。

现在把问题推广一下，假设有 a 头牛， b 辆车（门的总数为 $a+b$ ），在最终选择前主持人会替你打开 c 个有牛的门（ $1 \leq a \leq 10000$ ， $1 \leq b \leq 10000$ ， $0 \leq c < a$ ），输出“总是换门”的策略下，赢得车的概率。

【分析】

使用全概率公式。打开 c 个牛门后，还剩 $a-c$ 头牛，未开的门总数是 $a+b-c$ ，其中有

$a+b-c-1$ 个门可以换（称为“可选门”），换到门的概率就是“可选门”的总数除以“可选门中车门的个数”。

情况 1：一开始选了牛（概率 $a/(a+b)$ ），则可选门中车门有 b 个。这种情况的总概率为 $a/(a+b) * b/(a+b-c-1)$ 。

情况 2：一开始选了车（概率为 $b/(a+b)$ ），则可选门中车门只有 $b-1$ 个，概率为 $b/(a+b) * (b-1)/(a+b-c-1)$ 。

加起来得 $(ab+b(b-1))/((a+b)(a+b-c-1))$ 。

例题 10-11 条件概率 (Probability|Given, UVa11181)

有 n 个人准备去超市逛，其中第 i 个人买东西的概率是 P_i 。逛完以后你得知有 r 个人买了东西。根据这一信息，请计算每个人实际买了东西的概率。输入 n ($1 \leq n \leq 20$) 和 r ($0 \leq r \leq n$)，输出每个人实际买了东西的概率。

【分析】

“ r 个人买了东西”这个事件叫 E ，“第 i 个人买东西”这个事件为 E_i ，则要求的是条件概率 $P(E_i|E)$ 。根据条件概率公式， $P(E_i|E) = P(E_iE) / P(E)$ 。

$P(E)$ 依然可以用全概率公式。例如， $n=4, r=2$ ，有 6 种可能：1100, 1010, 1001, 0110, 0101, 0011，其中 1100 的概率为 $P_1 * P_2 * (1-P_3) * (1-P_4)$ ，其他类似，设置 $A[k]$ 表示第 k 个人是否买东西（1 表示买，0 表示不买），则可以用递归的方法枚举恰好有 r 个 $A[k]=1$ 的情况。

如何计算 $P(E_iE)$ 呢？方法一样，只是枚举的时候要保证第 $A[i]=1$ 。不难发现，其实可以用一次枚举就计算出所有的值。用 tot 表示上述概率之和， $\text{sum}[i]$ 表示 $A[i]=1$ 的概率之和，则答案为 $P(E_i)/P(E) = \text{sum}[i]/\text{tot}$ 。

例题 10-12 纸牌游戏 (Double Patience, NEERC 2005, UVa1637)

36 张牌分成 9 堆，每堆 4 张牌。每次可以拿走某两堆顶部的牌，但需要点数相同。如果有多种拿法则等概率的随机拿。例如，9 堆顶部的牌分别为 KS, KH, KD, 9H, 8S, 8D, 7C, 7D, 6H，则有 5 种拿法 (KS, KH), (KS, KD), (KH, KD), (8S, 8D), (7C, 7D)，每种拿法的概率均为 $1/5$ 。如果最后拿完所有牌则游戏成功。按顺序给出每堆牌的 4 张牌，求成功概率。

【分析】

用 9 元组表示当前状态，即每堆牌剩的张数，状态总数为 $5^9=1953125$ 。设 $d[i]$ 表示状态 i 对应的成功概率，则根据全概率公式， $d[i]$ 为后继状态的成功概率的平均值，按照动态规划的写法计算即可。

10.3 其他数学专题

10.3.1 递推

汉诺塔问题。假设有 A、B、C 3 个轴，有 n 个直径各不相同、从小到大依次编号为 1, 2, 3, ..., n 的圆盘按照上小下大的顺序叠放在 A 轴上。现要求将这 n 个圆盘移至 B 轴上并仍按同样顺序叠放，但圆盘移动时必须遵循下列规则：

- ❑ 每次只能移动一个圆盘，它必须位于某个轴的顶部。
- ❑ 圆盘可以插在 A、B、C 中的任一轴上。
- ❑ 任何时刻都不能将一个较大的圆盘压在较小的圆盘之上。

【分析】

这个问题看上去很容易，但当 n 稍大一点时，手工移动就开始变得困难起来。下面直接给出递归解法：首先，把前 $n-1$ 个圆盘放到 C 轴；接下来把 n 号圆盘放到 B 轴；最后，再把前 $n-1$ 个盘子放到 B 轴，如图 10-5 所示。

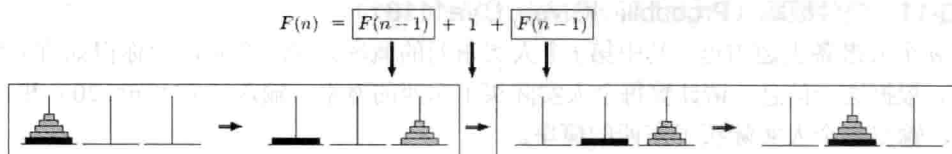


图 10-5 根据递归解法建立汉诺塔的递推关系

图 10-4 中还给出了 n 个圆盘所需步数 $f(n)$ 的递推式： $f(n)=2f(n-1)+1$ 。如果把 $f(n)$ 的值从小到大列出来，即 1,3,7,15,31,63,127,255…，你会发现其实有一个简单的表达式： $f(n)=2^n-1$ 。

用数学归纳法不难证明： $f(1)=1$ 满足等式。假设 $n=k$ 满足等式，即 $f(k)=2^k-1$ ，则 $n=k+1$ 时， $f(k+1)=2f(k)+1=2(2^k-1)+1=2^{k+1}-2+1=2^{k+1}-1$ 。因此 $n=k+1$ 也满足等式。由数学归纳法可知， n 取任意正整数均成立。

如果还不熟悉数学归纳法，其实从上面的证明过程已经能看出来其基本原理——其实它正是一种递归证明。只要边界处理好（ $f(1)=1$ 满足），递归时缩小规模（用 k 来证明 $k+1$ ），然后在“相信递归”（假设 $n=k$ 成立）的前提下证明即可。

提示 10-6：数学归纳法是一种利用递归的思想证明的方法。如果要讨论的对象具有某种递归性质（如正整数），可以考虑用数学归纳法。

Fibonacci 数列。先来考虑一个简单的问题：楼梯有 n 个台阶，上楼可以一步上一阶，也可以一步上两阶。一共有多少种上楼的方法？

这是一道计数问题。在没有思路时，不妨试着找规律。 $n=5$ 时，一共有 8 种方法：

5=1+1+1+1+1

5=2+1+1+1

5=1+2+1+1

5=1+1+2+1

5=1+1+1+2

5=2+2+1

5=2+1+2

5=1+2+2

其中有 5 种方法第 1 步走了 1 阶（灰色），3 种方法第 1 步走了 2 阶。没有其他可能了。假设 $f(n)$ 为 n 个台阶的走法总数，把 n 个台阶的走法分成两类。

第 1 类：第 1 步走 1 阶。剩下还有 $n-1$ 阶要走，有 $f(n-1)$ 种方法。

第2类：第1步走2阶。剩下还有 $n-2$ 阶要走，有 $f(n-2)$ 种方法。

这样，就得到了递推式： $f(n)=f(n-1)+f(n-2)$ 。不要忘记边界情况： $f(1)=1, f(2)=2$ 。当然，也可以认为边界是 $f(0)=f(1)=1$ 。把 $f(n)$ 的前几项列出：1, 1, 2, 3, 5, 8, …。

再例如，把雌雄各一的一对新兔子放入养殖场中。每只雌兔从第2个月开始每月产雌雄各一的一对新兔子。试问第 n 个月养殖场中共有多少对兔子？

还是先找找规律。

第1个月：一对新兔子 r_1 。用小写字母表示新兔子。

第2个月：还是一对新兔子，不过已经长大，具备生育能力了，用大写字母 R_1 表示。

第3个月： R_1 生了一对新兔子 r_2 ，一共两对。

第4个月： R_1 又生一对 r_3 ，一共3对。另外， r_2 长大了，变成 R_2 。

第5个月： R_1 和 R_2 各生一对，记为 r_4 和 r_5 ，共5对。此外， r_3 长成 R_3 。

第6个月： R_1 、 R_2 和 R_3 各生一对，记为 $r_6 \sim r_8$ ，共8对，同时 r_4 到 r_5 长大。

……

把这些数排列起来：1, 1, 2, 3, 5, 8, …，和刚才的一模一样！事实上，可以直接推导出递推关系 $f(n)=f(n-1)+f(n-2)$ ：第 n 个月的兔子由两部分组成，一部分是上个月就有的老兔子，一部分是上个月出生的新兔子。前一部分等于 $f(n-1)$ ，后一部分等于 $f(n-2)$ （第 $n-1$ 个月时具有生育能力的兔子数就等于第 $n-2$ 个月的兔子总数）。根据加法原理， $f(n)=f(n-1)+f(n-2)$ 。

提示 10-7：满足 $F_1=F_2=1, F_n=F_{n-1}+F_{n-2}$ 的数列称为 Fibonacci 数列，它的前若干项是 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, …。

再例如，有2行 n 列的长方形方格，要求用 n 个 1×2 的骨牌铺满。有多少种铺法？

考虑最左边一列的铺法。如果用一个骨牌直接覆盖，则剩下的 $2 \times (n-1)$ 方格有 $f(n-1)$ 种铺法；如果是用两个横向骨牌覆盖，则剩下的 $2 \times (n-2)$ 方格有 $f(n-2)$ 种方法，如图 10-6 所示。不难发现：第一列没有其他铺法，因此 $f(n)=f(n-1)+f(n-2)$ 。边界 $f(0)=1, f(1)=1$ ，恰好是 Fibonacci 数列。

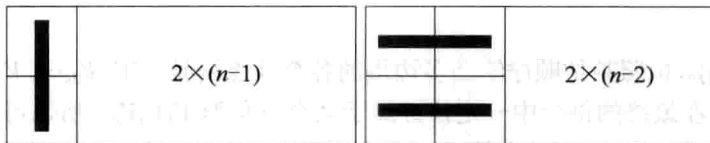


图 10-6 骨牌覆盖问题

这就是多数课本上讲解这道题目的方法，无须多说，因为重点并不在此。笔者曾想到过另一个解法，与各位读者分享：设第 i 列是纵向骨牌，则左边 $i-1$ 列和右边 $n-i$ 列各有 $f(i-1)$ 和 $f(n-i)$ 种铺法。根据乘法原理，一共有 $f(i-1)f(n-i)$ 种铺法。然后把 $i=1, 2, 3, \dots, n$ 的情形全部加起来，根据加法原理，有：

$$f(n) = f(0)f(n-1) + f(1)f(n-2) + \dots + f(n-1)f(0)$$

这个递推式对不对呢？聪明的读者也许已经看出，这个解法存在两个问题：

(1) 有遗漏。只考虑了第 $1, 2, 3, \dots, n$ 列是纵向骨牌的情形，但实际上可能所有的骨牌

都是横向的。当且仅当 n 为偶数时, 恰好有一种这样的方案。

(2) 有重复。根据“第 i 列有骨牌”对所有方案进行了分类, 但其实这些方案是有重叠的。例如, 第 1 列和第 2 列完全可以同时有骨牌。这些方案在递推式中被重复计算了。

既然如此, 这个思路是不是走入死胡同了呢? 不是的! 只要把刚才的推理变得严密起来, 同样可以得到一个正确的递推式: 根据从左到右第一条纵向骨牌的列编号分类。如果不存在, 当且仅当 n 为偶数时有一种方案; 当第一条纵向骨牌的列编号为 i 时, 意味着左边 $i-1$ 列必须全部是横向骨牌——当 i 为奇数时恰好有一个方案。而右边 $n-i$ 列则可以用任意铺法, 共 $f(n-i)$ 种。换句话说:

n 为偶数时, $f(n) = f(n-1) + f(n-3) + f(n-5) + \cdots + f(1) + 1$ (最后加上的就是“没有纵向骨牌”的情形)。

n 为奇数时, $f(n) = f(n-1) + f(n-3) + f(n-5) + \cdots + f(2) + f(0)$ 。

边界是 $f(0)=f(1)=1$ 。我们已经知道, 问题的答案应该是 Fibonacci 数列, 自然会对这个复杂的递推式产生怀疑: 它真的是正确的吗?

带着这个疑问, 笔者写了一个程序。结果出乎意料: 居然和 Fibonacci 数列一样! 事实上, 它确实是 Fibonacci 数列。Fibonacci 数列拥有很多有趣的性质, 有兴趣的读者可以在网上搜索更多相关资料。不管怎样, 这个“旧题新解”至少说明了两点:

(1) 一个数列可能有多个看上去完全不同的递推式。

(2) 即使是漏洞百出的解法也有可能通过“打补丁”的方式修改正确。

Catalan 数。 给一个凸 n 边形, 用 $n-3$ 条不相交的对角线把它分成 $n-2$ 个三角形, 求不同的方法数目。例如, $n=5$ 时, 有 5 种剖分方法, 如图 10-7 所示。

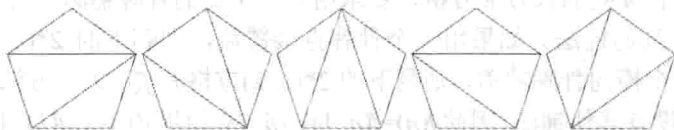


图 10-7 凸五边形的 5 种三角剖分

【分析】

设答案为 $f(n)$ 。按照某种顺序给凸多边形的各个顶点编号为 $V_1, V_2, \dots, V_n$ 。既然分成的是三角形, 边 V_1V_n 在最终的剖分中一定恰好属于某个三角形 $V_1V_nV_k$, 所以可以根据 k 进行分类。不难看出, 三角形 $V_1V_nV_k$ 的左边是一个 k 边形, 右边是一个 $n-k+1$ 边形 (如图 10-8 (a) 所示)。根据乘法原理, 包含三角形 $V_1V_nV_k$ 的方案数为 $f(k)f(n-k+1)$; 根据加法原理有:

$$f(n) = f(2)f(n-1) + f(3)f(n-2) + \cdots + f(n-1)f(2)$$

边界是 $f(2)=f(3)=1$ 。不难算出从 $f(3)$ 开始的前几项 f 值依次为: 1、2、5、14、42、132、429、1430、4862、16796。

提示 10-8: 在建立递推式时, 经常会用到乘法原理, 其核心是分步计数。如果可以把计数分成独立的两个步骤, 则总数量等于两步计数之乘积。

另一种思路是考虑 V_1 连出的对角线。对角线 V_1V_k 把凸 n 边形分成两部分, 一部分是 k 边形, 另一部分是 $n-k+2$ 边形 (如图 10-8 (b) 所示)。根据乘法原理, 包含对角线 V_1V_k

的凸多边形有 $f(k)f(n-k+2)$ 个。根据对称性, 考虑从 $V_2, V_3, \dots, V_n$ 出发的对角线也会有同样的结果, 因此一共有 $n(f(3)f(n-1)+f(4)f(n-2)+\dots+f(n-1)f(3))$ 个部分。

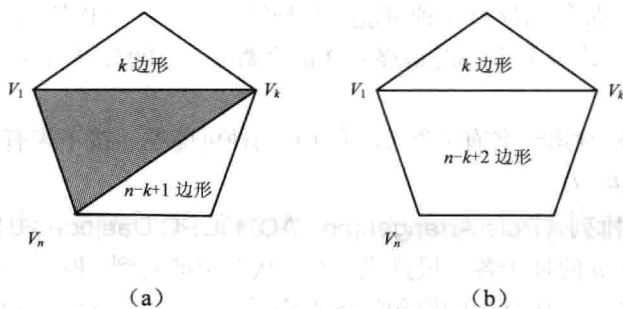


图 10-8 凸多边形三角剖分数目的两种递推方法

但这并不是正确答案, 因为同一个剖分被重复计算了多次! 不过这次不必去消除重复了, 因为这些重复很有规律: 每个方案恰好被计算了 $2n-6$ 次——有 $n-3$ 条对角线, 而考虑每条对角线的每个端点时均计算了一次。这样, 得到了 $f(n)$ 的第 2 个递推式:

$$f(n) = (f(3)(n-1) + f(4)f(n-2) + \dots + f(n-1)f(3)) \times n / (2n-6)$$

它和第一个递推式有几分相似, 但又不同。把 $n+1$ 代入第 1 个递推式后得到:

$$f(n+1) = f(2)f(n) + f(3)f(n-1) + f(4)f(n-2) + \dots + f(n-1)f(3) + f(n)f(2)$$

灰色部分是相同的! 根据第 2 个递推式, 它等于 $f(n) \cdot (2n-6)/n$, 把它和 $f(2)=1$ 一起代入上式得:

$$f(n+1) = f(n) + f(n) \cdot (2n-6)/n + f(n) = \frac{4n-6}{n} f(n)$$

这个递推式和前两个相比就简单多了。这个数列称为 Catalan 数, 也是常见的计数数列。

例题 10-13 危险的组合 (Critical Mass, UVa580)

有一些装有铀 (用 U 表示) 和铅 (用 L 表示) 的盒子, 数量均足够多。要求把 n ($n \leq 30$) 个盒子放成一行, 但至少要有 3 个 U 放在一起, 有多少种放法? 例如, $n=4, 5, 30$ 时答案分别为 3, 8 和 974791728。

【分析】

设答案为 $f(n)$ 。既然有 3 个 U 放在一起, 可以根据这 3 个 U 的位置分类——对, 根据前面的经验, 要根据“最左边的 3 个 U”的位置分类。假定是 $i, i+1$ 和 $i+2$ 这 3 个盒子, 则前 $i-1$ 个盒子不能有 3 个 U 放在一起的情况。设 n 个盒子“没有 3 个 U 放在一起”的方案数为 $g(n)=2^n - f(n)$, 则前 $i-1$ 个盒子的方案有 $g(i-1)$ 种。后面的 $n-i-2$ 个盒子可以随便选择, 有 2^{n-i-2} 种。根据乘法原理和加法原理, $f(n) = \sum_{i=1}^{n-2} g(i-1)2^{n-i-2}$ 。

遗憾的是, 这个推理是有瑕疵的。即使前 $i-1$ 个盒子内部不出现 3 个 U, 仍然可能和 $i, i+1$ 和 $i+2$ 组成 3 个 U。正确的方法是强制让第 $i-1$ 个盒子 (如果存在) 放 L, 则前 $i-2$ 个盒子内部不能出现连续的 3 个 U。因此 $f(n) = 2^{n-3} + \sum_{i=2}^{n-2} g(i-2)2^{n-i-2}$, 边界是 $f(0)=f(1)=f(2)=0$ 。

$g(0)=1, g(1)=2, g(2)=4$ 。注意上式中的 2^{n-3} 对应于 $i=1$ 的情况。

例题 10-14 比赛名次 (Race, UVa12034)

A、B 两人赛马，最终名次有 3 种可能：并列第一；A 第一 B 第二；B 第一 A 第二。输入 n ($1 \leq n \leq 1000$)，求 n 人赛马时最终名次的个数除以 10056 的余数。

【分析】

设答案为 $f(n)$ 。假设第一名有 i 个人，有 $C(n,i)$ 种可能性，接下来有 $f(n-i)$ 种可能性，因此答案为 $\sum C(n,i)f(n-i)$ 。

例题 10-15 杆子的排列 (Pole Arrangement, ACM/ICPC Daejeon 2012, UVa1638)

有高为 $1, 2, 3, \dots, n$ 的杆子各一根排成一行。从左边能看到 l 根，从右边能看到 r 根，求有多少种可能。例如，图 10-9 中的两种情况都满足 $l=1, r=2$ ($1 \leq l, r \leq n \leq 20$)。



图 10-9 杆子的排列

【分析】

设 $d(i,j,k)$ 表示让高度为 $1 \sim i$ 根杆子排成一行，从左边能看到 j 根，从右边能看到 k 根的方案数。为了方便起见，假定 $i \geq 2$ 。如何进行递推呢？首先尝试按照从小到大的顺序按照各个杆子。假设已经安排完高度为 $1 \sim i-1$ 的杆子，那么高度为 i 的杆子可能会挡住很多其他杆子，看上去很难写出递推式。

那么换一个思路：按照从大到小的顺序安排各个杆子。假设已经安排完高度为 $2 \sim i$ 的杆子，那么高度为 1 的杆子不管放哪里都不会挡住任何一根杆子。有如下 3 种情况。

情况 1：插到最左边，则从左边能看到它，从右边看不见（因为 $i \geq 2$ ）。

情况 2：如果插到最右边，则从右边能看到它，从左边看不见。

情况 3（有 $i-2$ 个插入位置）：插到中间，则不管从左边还是右边都看不见它。

在第一种情况下，高度为 $2 \sim i$ 的那些杆子必须满足：从左边能看到 $j-1$ 根，从右边能看到 k 根，因为只有这样，加上高度为 1 的杆子之后才是“从左边能看到 j 根，从右边能看到 k 根”。虽然状态 $d(i,j,k)$ 表示的是“让高度为 $1 \sim i$ 的杆子……”，而现在需要把高度为 $2 \sim i+1$ 的杆子排成一行，但是不难发现：其实杆子的具体高度不会影响到结果，只要有 i 根高度各不相同的杆子，从左从右看分别能看到 j 根和 k 根，方案数就是 $d(i,j,k)$ 。换句话说，情况 1 对应的方案数是 $d(i-1,j-1,k)$ 。类似地，情况 2 对应的方案数是 $d(i-1,j,k-1)$ ，而情况 3 对应的方案数是 $d(i-1,j,k)*(i-2)$ 。这样，就得到了如下递推式：

$$d(i,j,k) = d(i-1,j-1,k) + d(i-1,j,k-1) + d(i-1,j,k)*(i-2)$$

10.3.2 数学期望

数学期望。简单地说，随机变量 X 的数学期望 EX 就是所有可能值按照概率加权的和。

例如, 一个随机变量有 $1/2$ 的概率等于 1, $1/3$ 的概率等于 2, $1/6$ 的概率等于 3, 则这个随机变量的数学期望为 $1 \cdot 1/2 + 2 \cdot 1/3 + 3 \cdot 1/6 = 5/3$ 。在非正式场合中, 可以说这个随机变量“在平均情况下”等于 $5/3$ 。在解决和数学期望相关的题目时, 可以先考虑直接使用数学期望的定义求解: 计算出所有可能取值, 以及对应的概率, 最后求加权和, 如果遇到有困难, 则可以考虑使用下面两个工具:

期望的线性性质。有限个随机变量之和的数学期望等于每个随机变量的数学期望之和。例如, 对于两个随机变量 X 和 Y , $E(X+Y) = EX + EY$ 。

全期望公式。类似全概率公式, 把所有情况不重复、不遗漏地分成若干类, 每类计算数学期望, 然后把这些数学期望按照每类的概率加权求和。

例题 10-16 过河 (Crossing Rivers, ACM/ICPC Wuhan 2009, UVa12230)

你住在村庄 A, 每天需要过很多条河到另一个村庄 B 上班。B 在 A 的右边, 所有的河都在中间。幸运的是, 每条河上都有匀速移动的自动船, 因此每当到达一条河的左岸时, 只需等船过来, 载着你过河, 然后在右岸下船。你很瘦, 因此上船之后船速不变。

日复一日, 年复一年, 你问自己: 从 A 到 B, 平均情况下需要多长时间? 假设在出门时所有船的位置都是均匀随机分布。如果位置不是在河的端点处, 则朝向也是均匀随机。在陆地上行走的速度为 1。

输入 A 和 B 之间河的个数 n 、长度 D ($0 \leq n \leq 10$, $1 \leq D \leq 1000$), 以及每条河的左端点坐标离 A 的距离 p , 长度 L 和移动速度 v ($0 \leq p < D$, $0 < L \leq D$, $1 \leq v \leq 100$), 输出 A 到 B 时间的数学期望。输入保证每条河都在 A 和 B 之间, 并且相互不会重叠。

【分析】

用数学期望的线性。过每条河的时间为 L/v 与 $3L/v$ 的均匀分布, 因此期望过河时间为 $2L/v$ 。把所有 $2L/v$ 加起来, 再加上 $D - \text{sum}(L)$ 即可。

例题 10-17 糖果 (Candy, ACM/ICPC Chengdu 2012, UVa1639)

有两个盒子各有 n ($n \leq 2 \cdot 10^5$) 个糖, 每天随机选一个 (概率分别为 p , $1-p$), 然后吃一颗糖。直到有一天, 打开盒子一看, 没糖了! 输入 n, p , 求此时另一个盒子里糖的个数的数学期望。

【分析】

根据期望的定义, 不妨设最后打开第 1 个盒子, 此时第 2 个盒子有 i 颗, 则这之前打开过 $n+(n-i)$ 次盒子, 其中有 n 次取的是盒子 1, 其余 $n-i$ 次取的盒子 2, 概率为 $C(2n-i, n)p^{n+1}(1-p)^{n-i}$ 。注意 p 的指数是 $n+1$, 因为除了前面打开过 n 次盒子 1 之外, 最后又打开了一次。

这个概率表达式在数学上是正确的, 但是用计算机计算时需要小心: n 可能高达 20 万, 因此 $C(2n-i, n)$ 可能非常大, 而 p^{n+1} 和 $(1-p)^{n-i}$ 却非常接近 0。如果分别计算这 3 项再乘起来, 会损失很多精度。一种处理方式是利用对数, 设 $v1(i) = \ln(C(2n-i, n)) + (n+1)\ln(p) + (n-i)\ln(1-p)$, 则“最后打开第 1 个盒子”对应的数学期望为 $e^{v1(i)}$ 。

同理, 当最后打开的是第 2 个盒子, 对数为 $v2(i) = \ln(C(2n-i, n)) + (n+1)\ln(1-p) + (n-i)\ln(p)$, 概率为 $e^{v2(i)}$ 。根据数学期望的定义, 最终答案为 $\text{sum}\{i(e^{v1(i)} + e^{v2(i)})\}$ 。

例题 10-18 优惠券 (Coupons, UVa10288)

大街上到处在卖彩票, 一元钱一张。购买撕开它上面的锡箔, 你会看到一个漂亮的图

案。图案有 n 种, 如果你收集到所有 n ($n \leq 33$) 种彩票, 就可以得大奖。请问, 在平均情况下, 需要买多少张彩票才能得到大奖呢? 如 $n=5$ 时答案为 $137/12$ 。

【分析】

已有 k 个图案, 令 $s=k/n$, 拿一个新的需要 t 次的概率: $s^{t-1}(1-s)$; 因此平均需要的次数为 $(1-s)(1+2s+3s^2+4s^3+\dots) = (1-s)E$, 而 $sE = s+2s^2+3s^3+\dots = E-(1+s+s^2+\dots)$, 移项得

$$(1-s)E = 1+s+s^2+\dots = 1/(1-s) = n/(n-k)$$

换句话说, 已有 k 个图案: 平均拿 $n/(n-k)$ 次就可多搜集一个, 所以总次数为:

$$n(1/n+1/(n-1)+1/(n-2)+\dots+1/2+1/1)$$

10.3.3 连续概率

连续概率。简单地说, 随机变量 X 的数学期望 EX 就是所有可能值按照概率加权的和。例如, 一个随机变量有 $1/2$ 的概率等于 1, $1/3$ 的概率等于 2, $1/6$ 的概率等于 3, 则比变量随机。

例题 10-19 概率 (Probability, UVa11346)

在 $[-a, a] \times [-b, b]$ 区域内随机取一个点 P , 求以 $(0,0)$ 和 P 为对角线的长方形面积大于 S 的概率 ($a, b > 0, S \geq 0$)。例如 $a=10, b=5, S=20$, 答案为 23.35%。

【分析】

根据对称性, 只需要考虑 $[0, a] \times [0, b]$ 区域取点即可。面积大于 S , 即 $xy > S$ 。 $xy=S$ 是一条双曲线, 所求概率就是 $[0, a] \times [0, b]$ 中处于双曲线上面的部分。为了方便, 还是求曲线下面的面积, 然后用总面积来减, 如图 10-10 所示。

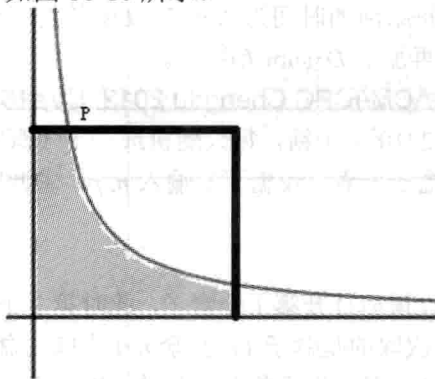


图 10-10 双曲线所围面积

设双曲线和区域 $[0, a] \times [0, b]$ 左边的交点 P 是 $(S/b, b)$, 因此积分就是:

$$S + S \int_{S/b}^a \frac{1}{x} dx$$

查得 $1/x$ 的原函数是 $\ln(x)$, 因此积分部分就是 $\ln(a) - \ln(S/b) = \ln(ab/S)$ 。设面积为 m , 则答案为 $(m - S \cdot \ln(m/S)) / m$ 。

注意这样做有个前提, 就是双曲线和所求区域相交。如果 $S > ab$, 则概率应为 0; 而如果 S 太接近 0, 概率应直接返回 1, 否则计算 $\ln(m/S)$ 时可能会出错。

例题 10-20 你想当 2^n 元富翁吗? (So you want to be a 2^n -aire?, UVa10900)

在一个电视娱乐节目中,你一开始有 1 元钱。主持人会问你 n 个问题,每次你听到问题后有两个选择:一是放弃回答该问题,退出游戏,拿走奖金;二是回答问题。如果回答正确,奖金加倍;如果回答错误,游戏结束,你一分钱也拿不到。如果正确地回答完所有 n 个问题,你将拿走所有的 2^n 元钱,成为 2^n 元富翁。

当然,回答问题是有风险的。每次听到问题后,你可以立刻估计出答对的概率。由于主持人会随机问问题,你可以认为每个问题的答对概率在 t 和 1 之间均匀分布。输入整数 n 和实数 t ($1 \leq n \leq 30, 0 \leq t \leq 1$),你的任务是求出在最优策略下,拿走的奖金金额的期望值。这里的最优策略是指让奖金的期望值尽量大。

【分析】

假设你刚开始游戏,如果直接放弃,奖金为 1;如果回答,期望奖金是多少呢?不仅和第 1 题的答对概率 p 相关,而且和答后面的题的情况相关。即:

选择“回答第 1 题”后的期望奖金 $= p * \text{答对 1 题后的最大期望奖金}$

注意,上式中“答对 1 题后的最大期望奖金”和这次的 p 无关,这提示我们用递推的思想,用 $d[i]$ 表示“答对 i 题后的最大期望奖金”,再加上“不回答”时的情况,可以得到:若第 1 题答对概率为 p ,期望奖金的最大值 $= \max\{2^0, p * d[1]\}$

这里故意写成 2^0 ,强调这是“答对 0 题后放弃”所得到的最终奖金。

上述分析可以推广到一般情况,但是要注意一点:到目前为止,一直假定 p 是已知的,而 p 实际上并不固定,而是在 $t \sim 1$ 内均匀分布。根据连续概率的定义, $d[i]$ 在概念上等于 $\max\{2^i, p * d[i+1]\}$ 在 $p=t \sim 1$ 上的积分。不要害怕“积分”二字,因为虽然在概念上这是一个积分,但是落实到具体的解法上,仍然只需要基础知识。

因为有 $\max$ 函数的存在,需要分两种情况讨论,即 $p * d[i+1] < 2^i$ 和 $p * d[i+1] \geq 2^i$ 两种情况。令 $p_0 = \max\{t, 2^i / d[i+1]\}$ (加了一个 $\max$ 是因为根据题目, $p \geq t$), 则:

□ $p < p_0$ 时, $p * d[i+1] < 2^i$, 因此“不回答”比较好,期望奖金等于 2^i 。

□ $p \geq p_0$ 时,“回答”比较好,期望奖金等于 $d[i]$ 乘以 p 的平均值 ($d[i]$ 作为常数被“提出来”了), 即 $(1+p_0)/2 * d[i+1]$ 。

在第一种情况中, p 的实际范围是 $[t, p_0]$, 因此概率为 $p_1 = (p_0 - t) / (1 - t)$ 。根据全期望公式, $d[i] = 2^i * p_1 + (1+p_0)/2 * d[i+1] * (1-p_1)$ 。

边界是 $d[n] = 2^n$, 逆向递推出 $d[0]$ 就是本题的答案。

例题 10-21 多边形 (Polygon, UVa11971)

有一根长度为 n 的木条,随机选 k 个位置把它们切成 $k+1$ 段小木条。求这些小木条能组成一个多边形的概率。

【分析】

不难发现本题的答案与 n 无关。在一条直线上切似乎难以处理,可以把直线接成一个圆,多切一下,即在圆上随机选 $k+1$ 个点,把圆周切成 $k+1$ 段。根据对称性,两个问题的答案相同。

新问题就要容易处理得多了:“组不成多边形”的概率就是其中一个小木条至少跨越了半个圆周的概率。设这个最长的小木条从点 i 开始逆时针跨越了至少半个圆周,则其他所

有点都在这半个圆周之外，如图 10-11 所示的灰色部分。

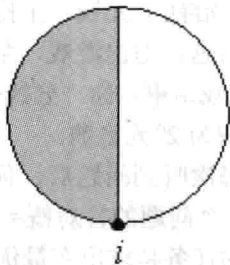


图 10-11 木条逆时针跨越所成形状

除了点 i 之外其他每个点位于灰色部分的概率均为 $1/2$ ，因此总概率为 $1/2^k$ 。点 i 的取法有 $k+1$ 种，因此“组不成多边形”的概率为 $(k+1)/2^k$ ，能组成多边形的概率为 $1-(k+1)/2^k$ 。

10.4 竞赛题目选讲

例题 10-22 统计问题 (The Counting Problem, ACM/ICPC Shanghai 2004, UVa1640)

给出整数 a, b ，统计 $a, a+1, \dots, b$ 中，数字 $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$ 分别出现了多少次。 $1 \leq a, b \leq 10^8$ 。

【分析】

解决这类题目的第一步一般都是：令 $f_d(n)$ 表示 $0 \sim n$ 中数字 d 出现的次数，则所求的就是 $f_d(b) - f_d(a-1)$ 。例如，要统计 $0 \sim 234$ 中 4 的个数，可以分成几个区间，如表 10-2 所示。

表 10-2 $0 \sim 234$ 所划区间

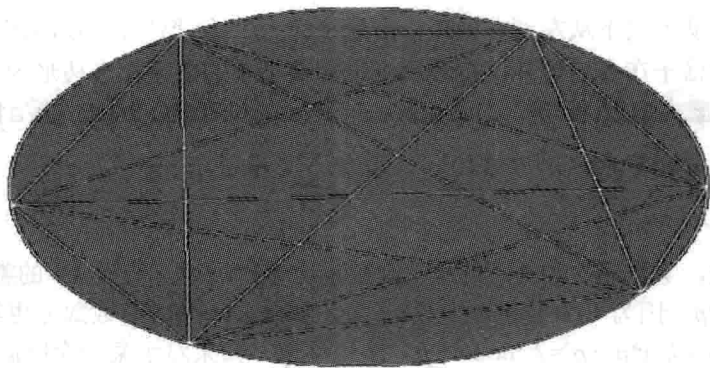
范 围	模 板 集
0~9	*
10~99	**
100~199	1**
200~229	20*, 21*, 22*
230~234	230, 231, 232, 233, 234

表 10-2 中的“模板”指的是一些整数的集合，其中字符“*”表示“任意字符”。例如， $1**$ 表示以 1 开头的任意 3 位数。因为后两个数字完全任意，所以“个位和十位”中每个数字出现的次数是均等的。换句话说，在模板 $1**$ 所对应的 100 个整数的 200 个“个位和十位”数字中， $0 \sim 9$ 各有 20 个。而这些数的百位总是 1，因此得到：模板 $1**$ 对应的 100 个整数包含数字 $0, 2 \sim 9$ 各 20 个，数字 1 有 120 个。

这样，只需把 $0 \sim n$ 分成若干个区间，算出每个区间中各个模板所对应的整数包含每个数字各多少次，就能解决原问题了，细节留给读者思考。

例题 10-23 多少块土地 (How Many Pieces of Land?, UVa10213)

有一块椭圆形的地。在边界上选 n ($0 \leq n < 2^{31}$) 个点并两两连接得到 $n(n-1)/2$ 条线段。它们最多能把地分成多少个部分？如图 10-12 所示， $n=6$ 时最多能分成 31 份。

图 10-12 $n=6$ 时所划分的土地

【分析】

本题需要用到欧拉公式：在平面图中， $V-E+F=2$ ，其中 V 是顶点数， E 是边数， F 是面数。因此，只需要计算 V 和 E 即可（注意还要减去外面的“无限面”）。

不管是顶点还是边，计算时都要枚举一条从固定点出发（所以最后要乘以 n ）的对角线，它的左边有 i 个点，右边有 $n-2-i$ 个点。左右点的连线在这条对角线上形成 $i(n-2-i)$ 个交点，得到 $i(n-2-i)+1$ 条线段。每个交点被重复计算了 4 次，每条线段被重复计算了 2 次。

$$V = n + \frac{n}{4} \cdot \sum_{i=1}^{n-3} i \cdot (n-2-i)$$

$$E = n + \frac{n}{2} \cdot \sum_{i=1}^{n-3} (i \cdot (n-2-i) + 1)$$

本题还有一个有趣之处： $n=1 \sim n=6$ 时答案分别为 1、2、4、8、16、31。如果根据前 5 项“找规律”得到“公式” 2^{n-1} ，即就错了。

例题 10-24 ASCII 面积 (ASCII Area, NEERC 2011, UVa1641)

在一个 $h \times w$ ($2 \leq h, w \leq 100$) 的字符矩阵里用 “.”、“\” 和 “/” 画出一个多边形，计算面积。如图 10-13 所示，面积为 8。

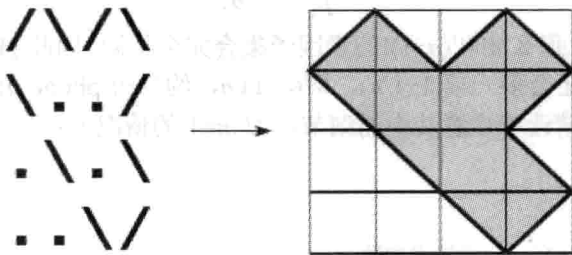


图 10-13 ASCII 面积

【分析】

这是一道和几何相关的题目，不过不需要高深的几何知识。每个格子要么全白，要么全黑，要么半白半黑，只要能准确地判断出来即可。字符 “\” 和 “/” 都是半白半黑，问题在于 “.” 到底是全白还是全黑。

解决方法是从上到下从左到右处理,沿途统计“/”和“\”。当这两个字符出现偶数次时说明接下来的格子在多边形外;奇数次则说明接下来的格子在多边形内。

例题 10-25 约瑟夫的数论问题 (Joseph's Problem, NEERC 2005, UVa1363)

输入正整数 n 和 k ($1 \leq n, k \leq 10^9$), 计算 $\sum_{i=1}^n k \bmod i$ 。

【分析】

被除数固定,除数逐次加1,直观上余数也应该有规律。假设 k/i 的整数部分等于 p , 则 $k \bmod i = k - i * p$ 。因为 $k/(i+1)$ 和 k/i 差别不大,如果 $k/(i+1)$ 的整数部分也等于 p , 则 $k \bmod (i+1) = k - (i+1) * p = k - i * p - p = k \bmod i - p$ 。换句话说,如果对于某一个区间 $i, i+1, i+2, \dots, j$, k 除以它们的商的整数部分都相同,则 k 除以它们的余数会是一个等差数列。

这样,可以在枚举 i 时把它所在的等差数列之和累加到答案中。这需要计算满足 $[k/j] = [k/i] = p$ 的最大 j 。

□ 当 $p=0$ 时这样的 j 不存在,所以等差序列一直延续到序列的最后。

□ 当 $p>0$ 时 j 为满足 $k/j \geq p$ 的最大 j , 即 $j \leq k/p$ 。除了首项之外的项数 $j-i \leq (k-i*p)/p = q/p$ 。

例题 10-26 帮帮 Tomisu (Help Mr. Tomisu, UVa11440)

给定正整数 N 和 M , 统计 2 和 $N!$ 之间有多少个整数 x 满足: x 的所有素因子都大于 M ($2 \leq N \leq 10^7, 1 \leq M \leq N, N-M \leq 10^5$)。输出答案除以 100000007 的余数。例如, $N=100, M=10$ 时答案为 43274465。

【分析】

因为 $M \leq N$, 所以 $N!$ 是 $M!$ 的整数倍。“所有素因子都大于 M ”等价于和 $M!$ 互素。另外,根据最大公约数的性质,对于 $k > M!$, k 与 $M!$ 互素当且仅当 $k \bmod M!$ 与 $M!$ 互素。这样,只需要求出“不超过 $M!$ 且与 $M!$ 互素的正整数个数”,再乘以 $N!/M!$ 即可。这样,问题的关键就是求出 $\text{phi}(M!)$ 。因为有多组数据,考虑用递推的方法求出所有的 $\text{phifac}(n) = \text{phi}(n!)$ 。由 phi 函数的公式:

$$\varphi(n) = n \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_k}\right)$$

如果 n 不是素数,那么 $n!$ 和 $(n-1)!$ 的素因子集合完全相同,因此 $\text{phifac}(n) = \text{phifac}(n-1) * n$; 如果 n 是素数,那么还会多一项 $(1-1/n)$, 即 $(n-1)/n$, 约分得 $\text{phifac}(n) = \text{phifac}(n-1) * (n-1)$ 。

核心代码如下(请读者注意其中的细节,如 $m=1$ 的情况):

```
int main() {
    int n, m;
    sieve(10000000); //筛法求素数
    phifac[1] = phifac[2] = 1; //请读者思考,为什么 phifac[1] 等于 1 而不是 0
    for(int i = 3; i <= 10000000; i++) //递推 phifac[i]=phi(i!)%MOD
        phifac[i] = (long long)phifac[i-1] * (vis[i] ? i : i-1) % MOD; //vis[i]
    为真⇔i 不是素数
```

```
while(scanf("%d%d", &n, &m) == 2 && n) {
```

```

int ans = phifac[m];
for(int i = m+1; i <= n; i++) ans = (long long)ans * i % MOD;
printf("%d\n", (ans-1+MOD)%MOD); //注意这里要减1, 因为题目从2开始统计
}
return 0;
}

```

例题 10-27 树林里的树 (Trees in a Wood, UVa10214)

在满足 $|x| \leq a$, $|y| \leq b$ ($a \leq 2000$, $b \leq 2000000$) 的网格中, 除了原点之外的整点 (即 x, y 坐标均为整数的点) 各种着一棵树。树的半径可以忽略不计, 但是可以相互遮挡。求从原点能看到多少棵树。如图 10-14 所示, 只有黑色的树可见。

【分析】

显然 4 个坐标轴上各只能看见一棵树, 所以可以只数第一象限 (即 $x > 0, y > 0$), 答案乘以 4 后加 4。第一象限的所有 x, y 都是正整数, 能看到 (x, y) , 当且仅当 $\gcd(x, y) = 1$ 。

由于 a 范围比较小, b 范围比较大, 一列一列统计比较快。第 x 列能看到的树的个数等于 $0 < y \leq b$ 的数中满足 $\gcd(x, y) = 1$ 的 y 的个数。可以分区间计算。

- $1 \leq y \leq x$: 有 $\phi(x)$ 个, 这是欧拉函数的定义。
- $x+1 \leq y \leq 2x$: 也有 $\phi(x)$ 个, 因为 $\gcd(x+i, x) = \gcd(x, i)$ 。
- $2x+1 \leq y \leq 3x$: 也有 $\phi(x)$ 个, 因为 $\gcd(2x+i, x) = \gcd(x, i)$ 。
-
- $kx+1 \leq y \leq b$: 直接统计, 需要 $O(x)$ 时间。

换句话说, 每次需要计算 $\phi(x)$ 和进行 $O(x)$ 次直接判断, 计算 $\phi(x)$ 需要 $O(x^{1/2})$ 时间, 而直接判断只需要 $O(1)$ 时间。再加上枚举 x 的所有 a 种可能, 总时间为 $O(a^2)$ 。

例题 10-28 (问题抽象) 高速公路 (Highway, ACM/ICPC CERC 2006, UVa1393)

有一个 n 行 m 列 ($1 \leq n, m \leq 300$) 的点阵, 问: 一共有多少条非水平非竖直的直线至少穿过其中两个点? 如图 10-15 所示, $n=2, m=4$ 时答案为 12, $n=m=3$ 时答案为 14。

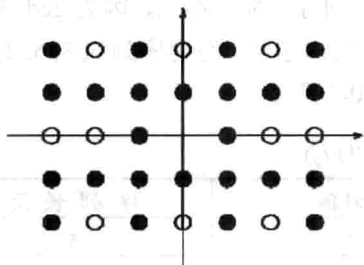


图 10-14 树林里的树



图 10-15 n 行 m 列点阵

【分析】

不难发现两个方向是对称的, 所以只统计 “\” 型的, 然后乘以 2。方法是枚举直线的包围盒大小 $a \times b$, 然后计算出包围盒可以放的位置。首先, 当 $\gcd(a, b) > 1$ 时肯定重复了, 如图 10-16 (a) 所示, 大包围盒 $a \times b$ 满足 $\gcd(a, b) > 1$, 在它的对角线和 $a \times b$ 的对角线是同一条

直线（其中 $a'=a/\gcd(a,b)$, $b'=b/\gcd(a,b)$ ）。

其次，如果放置位置不够靠左，也不够靠上，则它和它“左上方”的包围盒也重复了，如图 10-16 (b) 所示。

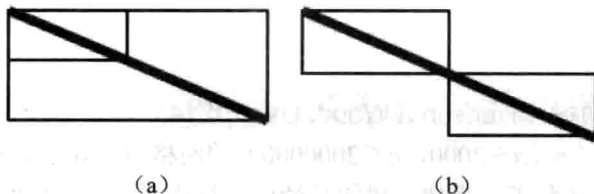


图 10-16 $\gcd(a,b)>1$ 时示意图

假定左上角坐标为 $(0,0)$ ，则对于左上角在 (x,y) 的包围盒，其“左上方”的包围盒的左上角为 $(x-a, y-b)$ 。这个“左上角”合法的条件是 $x-a \geq 0$ 且 $y-b \geq 0$ 。

包围盒本身不出界的条件是 $x+a \leq m-1$, $y+b \leq n-1$ ，一共有 $(m-a)(n-b)$ 个，而“左上方”有包围盒的情况，即 $a \leq x \leq m-a-1$ 且 $b \leq y \leq n-b-1$ ，有 $c = \max(0, m-2a) * \max(0, n-2b)$ 种放法。相减得到： $a*b$ 的包围盒有 $(m-a)(n-b)-c$ 种放法。

另外要注意应预处理保存所有 $\gcd$ ，而不是边枚举边算，否则会超时。

例题 10-29 魔法 GCD (Magical GCD, ACM/ICPC CERC 2013, UVa1642)

输入一个 n ($n \leq 100000$) 个元素的正整数序列 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^{12}$)，求一个连续子序列，使得该序列中所有元素的最大公约数与序列长度的乘积最大。例如，5 个元素的序列 30, 60, 20, 20, 20 的最优解为 {60, 20, 20, 20}，乘积为 $\gcd(60, 20, 20, 20) * 4 = 80$ 。

【分析】

本题看上去和第 8 章介绍的一些“传统算法题”很像，所以可试着沿用这样一个常见的框架：从左到右枚举序列的右边界 j ，然后快速求出左边界 $i \leq j$ ，使得 $\text{MGCD}(i, j)$ 最大，其中 $\text{MGCD}(i, j)$ 定义为 $\gcd(a_i, a_{i+1}, \dots, a_j) * (j-i+1)$ 。

如何快速求出 i 呢？好像那些“传统方法”（单调队列等）都用不上，因为 $\gcd$ 函数并没有很多“好用”的代数性质。怎么办？还是从数论的角度入手吧。考虑序列 5, 8, 6, 2, 6, 8，当 $j=5$ 时需要比较 $i=1, 2, 3, 4, 5$ 时的 $\text{MGCD}(i, j)$ ，如表 10-3 所示。

表 10-3 $j=5$ 时比较 i 的 $\text{MGCD}(i, j)$

i	$\gcd$ 表达式	$\gcd$ 值	序列长度
1	$\gcd(5, 8, 6, 2, 6)$	1	5
2	$\gcd(8, 6, 2, 6)$	2	4
3	$\gcd(6, 2, 6)$	2	3
4	$\gcd(2, 6)$	2	2
5	$\gcd(6)$	6	1

从下往上看， $\gcd$ 表达式里每次多一个元素，有时 $\gcd$ 不变，有时会变小，而且每次变小时一定是变成了它的某个约数（想一想，为什么）。换句话说，不同的 $\gcd$ 值最多只有

$\log_2 j$ 种! 当 gcd 值相同时, 序列长度越大越好, 所以可以把表 10-3 简化成表 10-4 中的形式。

表 10-4 简化表 10-3

gcd 值	1	2	6
i	1	2	5

因为表里只有 $\log_2 j$ 个元素, 所以可以依次比较每一个 i 对应的 $\text{MGCD}(i, j)$, 时间复杂度为 $O(\log j)$ 。下面考虑 j 从 5 变成 6 时, 这个表会发生怎样的变化。首先, 上述所有 gcd 值都要再和 $a_6=8$ 取 gcd, 即表 10-4 中第一行的 1, 2, 6 分别变成 $\text{gcd}(1, 8)=1$, $\text{gcd}(2, 8)=2$, $\text{gcd}(6, 8)=2$ 。然后要加入 $i=6$ 的序列, gcd 值为 8。由于相同的 gcd 值只需要保留 i 的最小值, 所以 $i=5$ 被删除, 最终得到如表 10-5 所示结果。

表 10-5 $i=5$ 被删除后的结果

gcd 值	1	2	6
i	1	2	8

上述过程需要删除 gcd 相同的重复元素, 但因为元素个数只有 $O(\log j)$ 个, 即使用二重循环比较, 时间效率也是很高的, 每次修改表 10-5 的时间复杂度为 $O((\log j)^2)$, 总时间复杂度为 $O(n(\log n)^2)$ 。但因为很难构造出每次表里都有接近 $\log_2 j$ 个元素的数据, 实际运行时间和时间复杂度为 $O(n \log n)$ 的算法相当。

10.5 训练参考

数学题目的特点是: 思维难度往往远大于编程难度。尽管如此, 也有一些程序实现细节不容忽视, 例如, 整数溢出和精度误差。本章的例题很多, 不过多数题目的难度不大, 重点在于帮助读者巩固相关的知识点。建议读者先学会所有不加星号的例题, 然后逐步弄懂有星号的例题。本章例题列表如表 10-6 所示。

表 10-6 例题列表

类 别	题 号	题目名称 (英文)	备 注
例题 10-1	UVa11582	Colossal Fibonacci Numbers!	模算术
例题 10-2	UVa12169	Disgruntled Judge	模算术
例题 10-3	UVa10375	Choose and Divide	唯一分解定理
例题 10-4	UVa10791	Minimum Sum LCM	唯一分解定理
例题 10-5	UVa12716	GCD XOR	数论
例题 10-6	UVa1635	Irrelevant Elements	组合数
例题 10-7	UVa10820	Send a Table	欧拉 phi 函数
例题 10-8	UVa1262	Password	编码解码问题
例题 10-9	UVa1636	Headshot	离散概率
例题 10-10	UVa10491	Cows and Cars	离散概率

续表

类 别	题 号	题目名称（英文）	备 注
例题 10-11	UVa11181	Probability Given	离散条件概率
例题 10-12	UVa1637	Double Patience	离散概率
例题 10-13	UVa580	Critical Mass	递推
例题 10-14	UVa12034	Race	递推
*例题 10-15	UVa1638	Pole Arrangement	递推
例题 10-16	UVa12230	Crossing Rivers	数学期望
例题 10-17	UVa1639	Candy	数学期望
例题 10-18	UVa10288	Coupons	数学期望
*例题 10-19	UVa11346	Probability	连续概率
*例题 10-20	UVa10900	So you want to be a 2 ⁿ -aire?	连续概率，数学期望
*例题 10-21	UVa11971	Polygon	连续概率
例题 10-22	UVa1640	The Counting Problem	数位统计
例题 10-23	UVa10213	How Many Pieces of Land?	欧拉公式、计数
例题 10-24	UVa1641	ASCII Area	多边形面积
例题 10-25	UVa1363	Joseph's Problem	数论，数列求和
*例题 10-26	UVa11440	Help Mr. Tomisu	欧拉 phi 函数
例题 10-27	UVa10214	Trees in a Wood	欧拉 phi 函数
例题 10-28	UVa1393	Highway	分类统计
例题 10-29	UVa1642	Magical GCD	综合题

本章的习题是本书中数量最多的，不过多数习题的难度不大，主要目的是巩固知识。因为大多数题目的描述比较简单，建议读者阅读所有题目，并选择感兴趣的题目思考。

习题 10-1 砌砖（Add Bricks in the Wall, UVa11040）

45 块石头按照如图 10-17 所示的方式排列，每块石头上有一个整数。

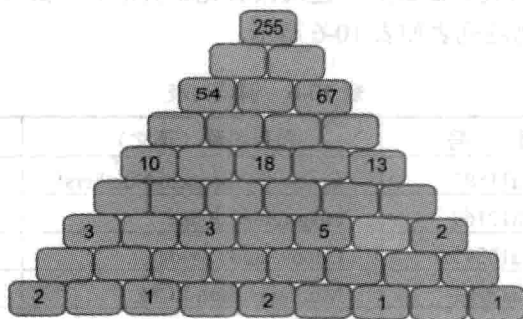


图 10-17 45 块石头排列方式

除了最后一行外，每个石头上的整数等于支撑它的两个石头上的整数之和。目前只有奇数行的左数奇数个位置上的数已知，你的任务是求出其余所有整数。输入保证有唯一解。

习题 10-2 勤劳的蜜蜂（Bee Breeding, ACM/ICPC World Finals 1999, UVa808）

如图 10-18 所示，输入两个格子的编号 a 和 b ($a, b \leq 10000$)，求最短距离。例如，19

和 30 的距离为 5（一条最短路是 19-7-6-5-15-30）。

习题 10-3 角度和正方形 (Angles and Squares, ACM/ICPC Beijing 2005, UVa1643)

如图 10-19 所示，第一象限里有一个角，把 n ($n \leq 10$) 个给定边长的正方形摆在这个角里（角度任意），使得阴影部分面积尽量大。

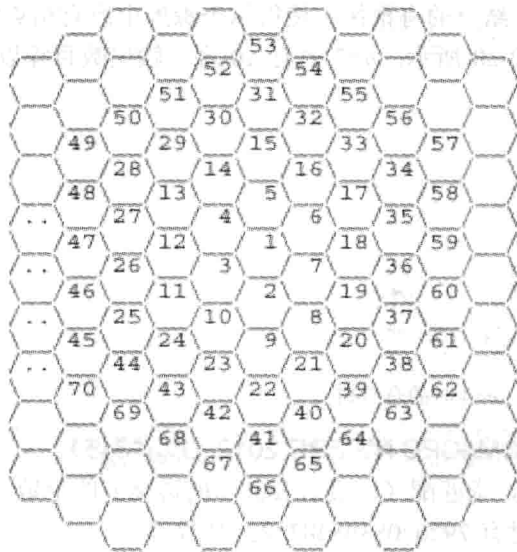


图 10-18 勤劳的蜜蜂问题示意图

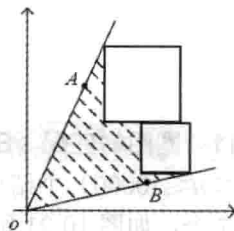


图 10-19 角度和正方形问题示意图

习题 10-4 素数间隔 (Prime Gap, ACM/ICPC Japan 2007, UVa1644)

输入一个整数 n ，求它后一个素数和前一个素数的差值。输入是素数时输出 0。 n 不超过 1299709（第 100000 个素数）。例如， $n=27$ 时输出 $29-23=6$ 。

习题 10-5 不同素数之和 (Sum of Different Primes, ACM/ICPC Yokohama 2006, UVa1213)

选择 K 个质数，使它们的和等于 N 。给出 N 和 K ($N \leq 1120$, $K \leq 14$)，问有多少种满足条件的方案？例如， $n=24$, $k=2$ 时有 3 种方案： $5+19=7+17=11+13=24$ 。注意，1 不是素数，因此 $n=k=1$ 时答案为 0。

习题 10-6 连续素数之和 (Sum of Consecutive Prime Numbers, ACM/ICPC Japan 2005, UVa1210)

输入整数 n ($2 \leq n \leq 10000$)，有多少种方案可以把 n 写成若干个连续素数之和？例如，41 可由 3 种方案： $2+3+5+7+11+13$, $11+13+17$ 和 41 写成。

习题 10-7 几乎是素数 (Almost Prime Numbers, UVa10539)

输入两个正整数 L 、 U ($L \leq U < 10^{12}$)，统计区间 $[L, U]$ 的整数中有多少个数满足：它本身不是素数，但只有一个素因子。例如，4、27 都满足条件。

习题 10-8 完全 P 次方数 (Perfect Pth Powers, UVa10622)

对于整数 x ，如果存在整数 b 使得 $x=b^p$ ，则说 x 是一个完全 p 次方数。输入整数 n ，求出最大的整数 p ，使得 n 是完全 p 次方数。 n 的绝对值不小于 2，且 n 在 32 位带符号整数范围内。例如， $n=17$, $p=1$; $n=1073741824$, $p=30$; $n=25$, $p=2$ 。

习题 10-9 约数 (Divisors, UVa294)

输入两个整数 L 、 U ($1 \leq L \leq U \leq 10^9$, $U-L \leq 10000$), 统计区间 $[L, U]$ 的整数中哪一个的正约数最多。如果有多个, 输出最小值。

习题 10-10 统计有根树 (Count, Chengdu 2012, UVa1645)

输入 n ($n \leq 1000$), 统计有多少个 n 结点的有根树, 使得每个深度中所有结点的子结点数相同。例如, $n=4$ 时有 3 棵, 如图 10-20 所示; $n=7$ 时有 10 棵。输出数目除以 10^9+7 的余数。

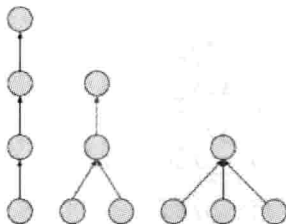


图 10-20 $n=4$ 时的有根树

习题 10-11 圈图的匹配 (Edge Case, ACM/ICPC NWERC 2012, UVa1646)

n ($3 \leq n \leq 10000$) 个结点组成一个圈, 求匹配 (即没有公共点的边集) 的个数。例如, $n=4$ 时有 7 个, 如图 10-21 所示, $n=100$ 时有 792070839848372253127 个。

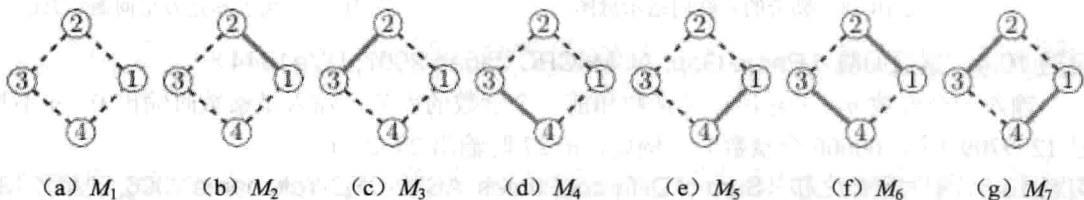


图 10-21 $n=4$ 时匹配的个数

习题 10-12 汉堡 (Burger, UVa557)

有 n 个牛肉堡和 n 个鸡肉堡给 $2n$ 个孩子吃。每个孩子在吃之前都要抛硬币, 正面吃牛肉堡, 反面吃鸡肉堡。如果剩下的所有汉堡都一样, 则不用抛硬币。求最后两个孩子吃到相同汉堡的概率。

习题 10-13 $H(n)$ ($H(n)$, UVa11526)

输入 n (在 32 位带符号整数范围内), 计算下面 C++ 函数的返回值:

```
long long H(int n){
    long long res = 0;
    for( int i = 1; i <= n; i=i+1 ){
        res = (res + n/i);
    }
    return res;
}
```

例如, $n=5$ 、10 时答案分别为 10 和 27。

习题 10-14 标准差 (Standard Deviation, UVa10886)

下面是一个随机数发生器。输入 seed 的初始值, 你的任务是求出它得到的前 n 个随机数标准差, 保留小数点后 5 位 ($1 \leq n \leq 10000000$, $0 \leq \text{seed} < 2^{64}$)。

```
unsigned long long seed;
long double gen()
{
    static const long double Z = (long double)1.0 / (1LL<<32);
    seed >>= 16;
    seed &= (1ULL << 32) - 1;
    seed *= seed;
    return seed * Z;
}
```

习题 10-15 零和一 (Zeros and Ones, ACM/ICPC Dhaka 2004, UVa12063)

给出 n 、 k ($n \leq 64$, $k \leq 100$), 有多少个 n 位 (无前导 0) 二进制数的 1 和 0 一样多, 且值为 k 的倍数?

习题 10-16 计算机变换 (Computer Transformations, ACM/ICPC SEERC 2005, UVa1647)

初始串为一个 1, 每一步会将每个 0 改成 10, 每个 1 改成 01, 因此 1 会依次变成 01, 1001, 01101001, ... 输入 n ($n \leq 1000$), 统计 n 步之后得到的串中, “00” 这样的连续两个 0 出现了多少次。

习题 10-17 H-半素数 (Semi-prime H-numbers, UVa11105)

所有形如 $4n+1$ (n 为非负整数) 的数叫 H 数。定义 1 是唯一的单位 H 数, H 素数是指本身不是 1, 且不能写成两个不是 1 的 H 数的乘积。H-半素数是指能写成两个 H 素数的乘积的 H 数 (这两个数可以相同也可以不同)。例如, 25 是 H-半素数, 但 125 不是。

输入一个 H 数 h ($h \leq 1000001$), 输出 $1 \sim h$ 之间有多少个 H-半素数。

习题 10-18 一个研究课题 (A Research Problem, UVa10837)

输入正整数 m ($m \leq 10^8$), 求最小的正整数 n , 使得 $\phi(n)=m$ 。输入保证 n 小于 200000000。

习题 10-19 蹦极 (Bungee Jumping, UVa10868)

James Bond 为了摆脱敌人的追击, 逃到了一座桥前。桥上正好有一条蹦极绳, 于是他打算把它拴到腿上, 纵身跳下桥, 落地后切断绳子, 继续逃生。已知绳子的正常长度为 l , Bond 的体重为 w , 桥的高度为 s , 你的任务是替 James Bond 判断能否用这种方法逃生。

当从桥上跳下后, 绳子绷紧前 Bond 将做自由落体运动 (重力按 $9.81w$ 计), 而绷紧后绳子会有向上的拉力, 大小为 $k \cdot \Delta l$, 其中 Δl 为绳子当前长度和正常长度之差。当且仅当 Bond 可以到达地面, 且落地速度不超过 10 米/秒时, 才认为他安全着落。

输入每组数据包含 4 个非负整数 k, l, s, w ($s < 200$)。对于每组数据, 如果可以安全着地, 输出 “James Bond survives.”, 如果到不了地面, 输出 “Stuck in the air.”, 如果到达地面速度太快, 输出 “Killed by the impact.”

习题 10-20 商业中心 (Business Center, NEERC 2009, UVa1648)

商业中心是一幢无限高的大楼。在一楼有 m 座电梯，每座电梯只有两个键：上、下。对于第 i 座电梯，每按一次“上”会往上走 u_i 层楼，每按一次“下”会往下走 d_i 层楼。你的任务是从一楼开始选一个电梯，恰好按 n 次按钮，到达一个尽量低（一楼除外）的楼层。中途不能换乘电梯。 $1 \leq n \leq 1000000$, $1 \leq m \leq 2000$, $1 \leq u_i, d_i \leq 1000$ 。

习题 10-21 二项式系数 (Binomial coefficients, ACM/ICPC NWERC 2011, UVa1649)

输入 m ($2 \leq m \leq 10^{15}$)，求所有的 (n, k) 使得 $C(n, k) = m$ 。输出按照 n 升序排列，当 n 相同时 k 按升序排列。

习题 10-22 飞机环球 (Planes Around the World, UVa10640)

有一种飞机，加满油能环游地球 a/b 圈。如果要使得一架飞机能够环游地球一圈，那么必须要使用其他若干架同种飞机，在某处为它空中加油。

假设 $a = 1$, $b = 2$, 5 架飞机可以环游。

首先 3 架飞机一起从 A 走到 C，飞机 3 给另外两架加满油，然后开始返程。当飞机 1 和 2 到达 D 的同时飞机 3 回到 A。然后飞机 2 给飞机 1 加满油，回到 A 点。

接下来，飞机 4 和 5 逆时针出发，其中飞机 4 在 F 处等待，飞机 5 在 E 处等待，直到飞机 1 到达 E。然后飞机 5 给飞机 1 加油，使得二者都能恰好飞到 F。然后飞机 4 给飞机 1 和飞机 5 加油，三者都恰好飞回 A，如图 10-22 所示。

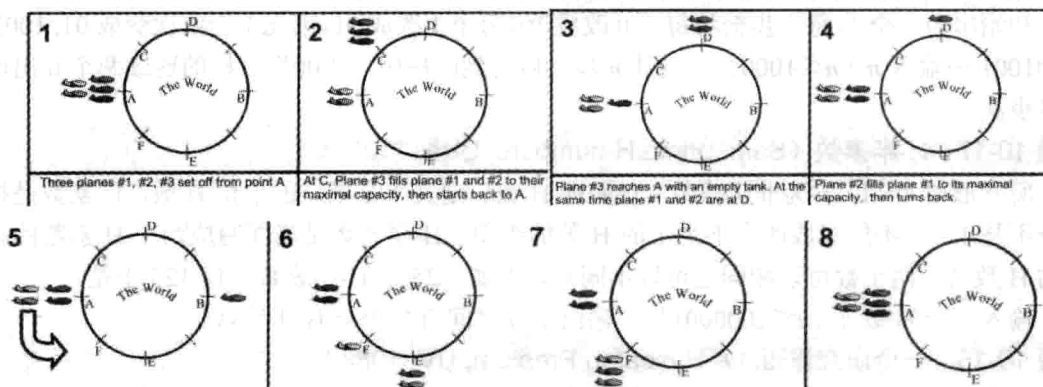


图 10-22 飞机环球问题示意图

假设：

- 只有飞机 1 环游地球。
- 有 A 架飞机和飞机 1 同时出发，同向飞行，称为正向飞机。每艘正向飞机都在某个位置处为其他飞机加油，然后折返。
- 有 B 架飞机于不同时间反向出发，称为反向飞机。每架反向飞机会停在一个地方等待飞机 1（及其他同行飞机）。等到之后为其他飞机加油，然后折返。
- 除了飞机 1 之外的其他飞机恰好为其他飞机加一次油，使得每个其他飞机得到相同多的油量。

输入 a, b ，输出最少需要时用多少架飞机才能完成环游地球。例如 $a = 1$, $b = 2$ 时需要

5 架。无解输出 -1。

习题 10-23 Hendrie 序列 (Hendrie Sequence, UVa10479)

Hendrie 序列是一个自描述序列, 定义如下:

□ $H(1)=0$ 。

□ 如果把 H 中的每个整数 x 变成 x 个 0 后面跟着 $x+1$, 则得到的序列仍然是 H (只是少了第一个元素)。

因此, H 序列的前几项为: $0, 1, 0, 2, 1, 0, 0, 3, 0, 2, 1, 1, 0, 0, 0, 4, 1, 0, 0, 3, 0, \dots$ 输入正整数 n ($n < 2^{63}$), 求 $H(n)$ 。

习题 10-24 幂之和 (Sum of Powers, UVa766)

对于正整数 k , 可以定义 k 次方和:

$$S_k(n) = \sum_{i=1}^n i^k$$

可以把它写成下面的形式。当 M 取最小可能的正整数时, 所有系数 a_i 都是确定的。

$$S_k(n) = \frac{1}{M} (a_{k+1}n^{k+1} + a_k n^k + \dots + a_1 n + a_0)$$

输入 k ($0 \leq k \leq 20$), 输出 $M, a_{k+1}, a_k, \dots, a_1, a_0$ 。例如, $k=2$, 输出 $6, 2, 3, 1, 0$ 。

习题 10-25 因子 (Factors, ACM/ICPC World Finals 2013, UVa1575)

算术基本定理: 每一个大于 1 的正整数都有唯一的方式写成若干个素数的乘积。不过如果允许把这些素数重排, 就有多重表示方式:

$$10 = 2 * 5 = 5 * 2, 20 = 2 * 2 * 5 = 2 * 5 * 2 = 5 * 2 * 2$$

令 $f(k)$ 为正整数 k 的写法个数, 如 $f(10)=2$, $f(20)=3$ 。对于正整数 n , 可以证明一定有整数 k 使得 $f(k)=n$ 。你的任务是求出最小的 k 。 $n < 2^{63}$ 。

习题 10-26 方形花园 (Square Garden, UVa12520)

在 $L \times L$ ($L \leq 10^6$) 网格里涂色 n ($n \leq L^2$) 个格子, 要求涂色格子的轮廓线周长尽量大。例如, 图 10-22 中为 $L=3$, $n=8$ 的两组解, 图 10-23 (a) 的周长为 16, 图 10-23 (b) 的周长为 12。

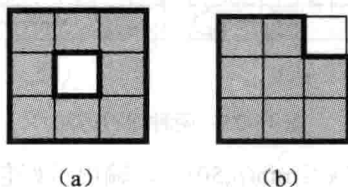


图 10-23 $L=3$, $n=8$ 的两组解

习题 10-27 互联 (Interconnect, ACM/ICPC NEERC 2006, UVa1390)

输入 n 个点 m 条边的无向图 G ($n \leq 30$, $m \leq 1000$)。每次随机加一条非自环的边 (u, v) (加完后可以出现重边)。添加每条边的概率是相等的, 求使 G 连通的期望操作次数。

习题 10-28 数字串 (Number String, ACM/ICPC Changchun 2011, UVa1650)

每个排列都可以算出一个特征, 即从第二个数开始每个数和前面一个数相比是增加(I)

还是减少(D)。例如, $\{3,1,2,7,4,6,5\}$ 的特征是 DIIDID。输入一个长度为 $n-1$ ($2 \leq n \leq 1001$) 的字符串 (包含字符 I, D 和 ?), 统计 $1 \sim n$ 有多少个排列的特征和它匹配 (其中 ? 表示 I 和 D 都符合)。输出答案除以 1000000007 的余数。

习题 10-29 名次表的变化 (Fantasy Cricket, UVa11982)

如图 10-24 所示为一个足球比赛的名次表, 给出了每个队伍相对上一轮的排名变化。例如:

Rank		Manager
1	▲	A
2	▼	B
3	▲	C
4	▼	D

图 10-24 足球比赛名次表

这代表队伍 A 的名次提高了, B 降低了, C 提高了, D 降低了。用 U 表示排名上升, D 表示降低, E 表示不变, 则上表可以用 UDUD 表示。经过计算可知, 上一轮的名次表有两种可能: BADC 和 BDAC (假定本轮和上一轮的名次都没有并列)。

输入这样一个 UDE 组成的序列 (长度不超过 1000), 求上一轮名次有多少种可能。输出答案除以 10^9+7 的余数。

习题 10-30 守卫 (Guard, ACM/ICPC Dhaka 2011, UVa12371)

在 $n \times n$ 棋盘上放 $2n$ 个守卫, 使得每行每列均恰好有两个守卫, 且一个格子里最多只有一个守卫。如图 10-25 所示是两种方法, 其中图 10-25 (a) 的守卫形成一个大圈, 图 10-25 (b) 中形成两个小圈。

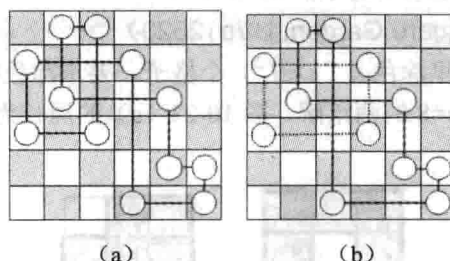


图 10-25 两种守卫方法

输入 n, k ($2 \leq n \leq 10^5, 1 \leq k \leq \min(n, 50)$), 输出恰好包含 k 个圈的方案总数。例如, $n=2, k=1$ 答案为 1; $n=3, k=1$, 答案为 6; $n=4, k=1$, 答案为 72; $n=4, k=2$, 答案为 18。

习题 10-31 守卫 II (Guards II, ACM/ICPC Dhaka 2012, UVa12590)

在 n 行 m 列的棋盘里放 k 个车, 使得边界格子都被攻击到。输出方案总数除以 10^9+7 的余数。 $n, m, k \leq 100$ 。输入最多包含 20000 组数据。

习题 10-32 汉诺塔 (Hanoi Towers, ACM/ICPC NEERC 2007, UVa1414)

Hanoi 塔问题有一种构造解法: 把 6 种移动 (AB, AC, BA, BC, CA, CB) 排序后选择第一个能用的操作, 前提是不能连续移动同一个盘子。给出 n ($n \leq 30$) 和 6 种移动的顺序, 求

解 Hanoi 问题的步数。最终所有盘子可以都在 B 也可以都在 C。例如, 对于 $n=2$, 排序为 AB, BA, CA, BC, CB, AC, 一共需要 5 步。

习题 10-33 二元运算 (Binary Operation, ACM/ICPC NEERC 2010, UVa1651)

给定正整数 $a \leq b$, 你的任务是计算 $a \otimes (a+1) \otimes (a+2) \otimes \cdots \otimes (b-1) \otimes b$ 的值, 其中 $a \otimes b$ 的计算方法是这样的: 首先, 如果 a 和 b 的位数不同, 位数较少的一个前面补 0; 然后逐位执行 $\odot$ 操作。例如, 当 $\odot$ 表示“加起来模 10”时, $5566 \otimes 239$ 的计算方法如下:

$$\begin{array}{r} \otimes \quad 5566 \\ \otimes \quad 239 \\ \hline \quad ??? \end{array} \rightarrow \begin{array}{r} \otimes \quad 5566 \\ \otimes \quad 0239 \\ \hline \quad ??? \end{array} \rightarrow \begin{array}{r} \quad 5 \quad 5 \quad 6 \quad 6 \\ \odot \quad 0 \quad \odot \quad 2 \quad \odot \quad 3 \quad \odot \quad 9 \\ \hline \quad 0 \quad 0 \quad 8 \quad 4 \end{array} \rightarrow \begin{array}{r} \otimes \quad 5566 \\ \otimes \quad 0239 \\ \hline \quad 0084 \end{array} \rightarrow \begin{array}{r} \otimes \quad 5566 \\ \otimes \quad 239 \\ \hline \quad 84 \end{array}$$

操作符 $\otimes$ 是左结合的, 因此 $a \otimes (a+1) \otimes (a+2) \otimes \cdots \otimes (b-1) \otimes b$ 从左到右计算即可。

输入 $\odot$ 的运算表 (一个 10×10 矩阵, 表示 $0 \odot 0, 0 \odot 1, \dots, 9 \odot 9$ 的结果, 其中 $0 \odot 0$ 保证为 0) 和 a, b ($0 \leq a \leq b \leq 10^{18}$) 的值, 输出所求结果。

习题 10-34 记住密码 (Password Remembering, ACM/ICPC Dhaka 2009, UVa12212)

输入正整数 A, B ($A \leq B < 2^{64}$), 求有多少个整数 n 满足: n 在 A 和 B 之间 (即 $A \leq n \leq B$), 且 n 翻转之后也在 A 和 B 之间。1203 翻转以后为 3021, 1050 翻转以后是 501。

习题 10-35 Fibonacci 单词 (Fibonacci Word, ACM/ICPC World Finals 2012, UVa1282)

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n \geq 2 \end{cases}$$

输入非空 01 串 p 和 n ($0 \leq n \leq 100$), 求 p 在 $F(n)$ 中出现几次。 p 的长度不超过 100000。

习题 10-36 Fibonacci 进制 (Fibonacci System, ACM/ICPC NEERC 2008, UVa1652)

每个正整数都可以写成 $N = a_n F_n + a_{n-1} F_{n-1} + \cdots + a_1 F_1$, 其中 $a_n = 1$, F_i 就是第 i 个 Fibonacci 数 ($F_0 = F_1 = 1$, $F_i = F_{i-1} + F_{i-2}$), 然后用 $a_n a_{n-1} \cdots a_2 a_1$ 作为 N 的 Fibonacci 进制表示。规定不能出现两个连续的 1。例如, 1~7 的 Fibonacci 进制表示分别为: 1, 10, 100, 101, 1000, 1001, 1010。

把所有自然数的 Fibonacci 进制表示拼起来, 会得到一个长长的串 110100101100010011010…。输入 nn ($n \leq 10^{15}$), 统计前 n 位有多少个 1。

习题 10-37 倍数问题 (Yet Another Multiple Problem, Chengdu 2012, UVa1653)

输入一个整数 n ($1 \leq n \leq 10000$) 和 m 个十进制数字, 找 n 的最小倍数, 其十进制表示中不含这 m 个数字中的任何一个。

提示: 需要建一张图, 结点 i 代表除以 n 的余数等于 i 。巧妙地利用第 6 章学过的 BFS 树可以简洁地解决这个问题。

习题 10-38 正多边形 (Regular Polygon, UVa10824)

给出圆周上的 n ($n \leq 2000$) 个点, 选出其中的若干个组成一个正多边形, 有多少种方法? 输出每行包含两个整数 S 和 F , 表示有 F 种选法得到正 S 边形。各行应按 S 从小到大排序。

习题 10-39 圆周上的三角形 (Circum Triangle, UVa11186)

在一个圆周上有 n ($n \leq 500$) 个点。不难证明, 其中任意 3 个点都不共线, 因此都可以

组成一个三角形。求这些三角形的面积之和。

习题 10-40 实验法计算概率 (Probability Through Experiments, ACM/ICPC Hatyai 2012, UVa12535)

输入圆的半径和圆上 n ($n \leq 20000$) 个点的极角, 任选 3 点能组成多少个锐角三角形?

习题 10-41 整数序列 (A Sequence of Numbers, ACM/ICPC Chengdu 2007, UVa1406)

输入 n 个整数, 执行 Q 个操作 ($n \leq 10^5$, $Q \leq 200000$)。有两种操作:

□ ADD d : 把所有数加上一个定值 d 。

□ QUERY i : 统计有多少个数的二进制表示法中第 i 位上是 1, 并输出。

习题 10-42 网格中的三角形 (Triangles in the Grid, UVa12508)

一个 n 行 m 列的网格有 $n+1$ 条横线和 $m+1$ 条竖线。任选 3 个点, 可以组成很多三角形。其中有多少个三角形的面积位于闭区间 $[A, B]$ 内? $1 \leq n, m \leq 200$, $0 \leq A < B \leq nm$ 。

习题 10-43 整数对 (Pair of Integers, ACM/ICPC NEERC 2001, UVa1654)

考虑一个不含前导 0 的正整数 X , 把它去掉一个数字以后得到另外一个数 Y 。输入 $X+Y$ 的值 N ($1 \leq N \leq 10^9$), 输出所有可能的等式 $X+Y=N$ 。例如, $N=34$ 有两个解: $31+3=34$; $27+7=34$ 。

习题 10-44 选整数 (K-Multiple Free Set, UVa11246)

给定正整数 k , 从 $1 \sim n$ 的整数中选出尽量多的整数, 使得没有一个整数是另一个整数的 k 倍。例如, $n=10$, $k=2$, 最多可以选 6 个: $1, 3, 4, 5, 7, 9$ 。 $1 \leq n \leq 10^9$, $2 \leq k \leq 100$ 。

习题 10-45 带符号二进制 (Power Signs, UVa11166)

每个整数都可以写成二进制。现将二进制变一下: 每个数位上可以是 0 和 1, 还可以是 -1。例如, 13 可以写成 $(1, 0, 0, -1, -1) = 2^4 - 2^1 - 2^0$ 。在这种进位制下, 正整数的表示方法不唯一, 例如, 7 可以写成 $(1, 1, 1)$ 或者 $(1, 0, 0, -1)$ 。你的任务是找一种非 0 数字最少的表示法。

输入每组数据第一行为用二进制表示的正整数 n ($n \leq 2^{5000}$), 保证不含前导 0。对于每组数据, 输出非 0 数字最小的表示法 (0 表示 0, + 表示 1, - 表示 -1)。如果有多解, 输出字典序最小的。

习题 10-46 抽奖 (Honorary Tickets, UVa11895)

在一次抽奖活动中, 有 n ($1 \leq n \leq 10^5$) 个抽奖箱, 其中第 i 个箱子里有 t_i ($t_i > 0$) 个信封, 其中 l_i 个里面有奖。所有人依次抽奖 (即自主选择一个抽奖箱, 然后随机抽一个信封), 每次抽完后的空信封放回去。假设每个人都知道上述数据, 并且足够聪明, 求第 k 个人抽到奖的概率 (用最简分数表示, 保证分子和分母都在 32 位带符号整数范围内)。注意, 每个人抽到奖之后只会默默地将它拿出, 其他人并不会知道, 因此不会改变既定的策略。

习题 10-47 随机数 (Randomness, UVa11429)

你有一个随机数发生器 (RNG), 可以得到 $1 \sim R$ ($2 \leq R \leq 1000$) 之间的随机整数 (每个整数的概率均为 $1/R$)。现在你希望用它在 N ($2 \leq N \leq 1000$) 个事件中随机选择一个, 使得事件 i 的概率 P_i 等于给定的有理数 a_i/b_i ($1 \leq a_i < b_i \leq 1000$)。你的任务是设计一个 RNG 使用算法, 使得对 RNG 的调用次数的数学期望尽量小。可以多次使用这个 RNG。

例如, 当 $R=2$, $N=4$, $P_1=P_2=P_3=P_4=1/4$ 时, 则只需调用两次 RNG, 一共有 4 种可能的结果, 分别对应一个事件。

习题 10-48 考试 (Exam, ACM/ICPC Chengdu 2012, UVa1655)

设 $f(x)$ 为满足 $ab|x$ 的 (a,b) 个数。输入 n ($1 \leq n \leq 10^{11}$)，求 $f(1)+f(2)+\cdots+f(n)$ 。例如， $f(1)=1$ ， $f(2)=3$ ， $f(3)=3$ ， $f(4)=6$ ， $f(5)=3$ ， $f(6)=9$ （即 $(1,1), (1,2), (2,1), (1,3), (3,1), (1,6), (6,1), (2,3), (3,2)$ ），因此 $n=6$ 时输出 25。

习题 10-49 指数塔 (Exponential Towers, ACM/ICPC NWERC 2013, UVa1656)

用“ $\wedge$ ”来表示指数运算，即 $a^b = a^b$ ，例如， $256 = 2^{2^3} = 4^{2^2}$ （注意“ $\wedge$ ”是右结合的，即 2^{2^3} 表示 $2^{(2^3)}$ ）。定义 $a_1^{a_2^{a_3^{\cdots^{a_k}}}}$ 这样的表达式为“高度为 k 的指数塔”，其中 $k > 1$ ，且所有整数 $a_i > 1$ 。输入一个高度为 3 的指数塔 a^b^c ($1 \leq a, b, c \leq 9585$)，统计有多少个高度至少为 3 的指数塔的值等于 a^b^c 。注意，9585 这个常数可以保证输出小于 2^{63} 。

习题 10-50 排列 (Permutation, UVa11303)

输入一个长度为 m 的序列，每个元素均为 $1 \sim n$ 的正整数，并且不含相同元素。找出 $1 \sim n$ 的排列中有哪些排列包含输入子序列（不一定连续出现），求出字典序第 k 小的。例如，若输入子序列为 $1, 3, 2, n=4$ ，则一共有 4 个排列： $1, 3, 2, 4$ ； $1, 3, 4, 2$ ； $1, 4, 3, 2$ ； $4, 1, 3, 2$ ，它们的字典序分别为第 1, 2, 3, 4 小。 $1 \leq n \leq 250$ ， $1 \leq m \leq n$ 。

习题 10-51 游戏 (Game, ACM/ICPC ACM/ICPC NEERC 2003, UVa1657)

有这样一个游戏：裁判先公布一个正整数 n ($2 \leq n \leq 200$)，然后在 $1 \sim n$ 中选两个不同的整数 x 和 y ($x < y$)，把 $x+y$ 告诉 S 先生，把 $x*y$ 告诉 P 先生，然后依次循环 S 先生和 P 先生是否知道这两个数是几（总是先问 S 先生）。例如：

裁判： $n=10$ （然后悄悄告诉 S： $x+y=9$ ， $x*y=18$ ）。

S 先生：不知道 x 和 y 是多少。

P 先生：不知道 x 和 y 是多少。

S 先生：不知道 x 和 y 是多少。

P 先生：不知道 x 和 y 是多少。

S 先生：知道了。 $x=3$ ， $y=6$ 。

两人一共说了 m 次“不知道”后，下一个人算出了答案。已知 S 和 P 都非常聪明且精于心算，你的任务是根据 n 和 m ($0 \leq m \leq 100$) 计算出所有可能的 (x, y) 。

例如， $n=10$ ， $m=4$ 时有 3 个解： $(2,5), (3,6), (3,10)$ 。